Appunti di metodi numerici

l'arte della programmazione

Nicolò Mone

Qui di solito ci va la common creative license ma chi ce l'ha e chi se ne fotte

Qualcosa di commovente

Introduzione

Pisa, 24 marzo 2026 Nicolò Mone

Indice

Introduzione **5**

1 Numeri casuali **9**
1.0.1 Introduzione . . . 9
1.0.2 Generatori congruenziali lineari . . . 10
1.0.3 Combinazione di generatori . . . 11
1.0.4 Generatori basati su sistemi caotici . . . 12
1.0.5 Generatori come sistemi dinamici a molti stati . . . 13
1.1 Generazione di distribuzioni non uniformi . . . 13
1.1.1 Dal caso uniforme a una distribuzione arbitraria . . . 13
1.1.2 Metodo della funzione cumulativa inversa . . . 14
1.1.3 Trasformazioni in più dimensioni . . . 15
1.1.4 Generazione di distribuzioni gaussiane: metodo di Box-Muller . . . 15
1.1.5 Osservazione conclusiva su questa parte . . . 17
1.1.6 Rejection method . . . 17
1.1.7 Metodo del rifiuto per la distribuzione Gamma . . . 18
1.2 Integrazione Monte Carlo . . . 20

2 Ricerca degli zeri di una funzione **27**
2.1 Introduzione . . . 27
2.2 Metodo di bisezione . . . 28
2.2.1 Osservazioni sul metodo di bisezione . . . 29
2.3 Metodo di Newton-Raphson . . . 30
2.3.1 Analisi dell'errore . . . 30
2.3.2 Osservazioni sul metodo di Newton . . . 31
2.4 Metodi per polinomi . . . 31
2.5 Generalizzazione a più dimensioni . . . 32
2.6 Conclusioni . . . 33

3 Analisi numerica nell'integrazione **35**
3.1 Introduzione . . . 35
3.1.1 Formule di NewtonCotes su intervallo singolo . . . 36
3.1.2 Alcune formule locali . . . 36
3.1.3 Intervalli estesi . . . 37
3.1.4 Valutazione dellerrore e formula esatta dei trapezi . . . 40
3.1.5 Integrazione di Romberg . . . 40
3.1.6 Quadrature Gaussiane . . . 42
3.1.7 Integrali impropri . . . 44

4 Integrazione numerica delle Equazioni Differenziali Ordinarie (ODE) 47
4.0.1 Metodi multi-step 48
4.0.2 Metodi one-step 53
4.0.3 Equazioni stiff 58
4.0.4 Boundary Value Problems (BVP) 59
4.1 Equazioni differenziali alle derivate parziali (PDE) 61
4.1.1 Differenze finite 62
4.1.2 Flux-conservative PDEs 63
4.1.3 Una soluzione pratica 64

A Metodo delle differenze finite 69

B Algoritmo di Thomas 73

C Metodo di Jacobi 77
C.0.1 Errore 80

D Gauss-Siedel, SOR & LOR 85

E Alternate direction implicit 87

F Metodi del gradiente 89

Capitolo 1

Numeri casuali

1.0.1 Introduzione

In molte applicazioni di analisi numerica è necessario disporre di sequenze di numeri casuali. In realtà i computer non producono numeri casuali, generano sequenze *pseudo-casuali*, cioè successioni determinate da algoritmi deterministici che imitano alcune proprietà statistiche della casualità.

In linea di principio esistono algoritmi che sfruttano sorgenti fisiche per produrre numeri "realmente" (?) casuali, ma nella maggior parte delle applicazioni numeriche si utilizzano generatori pseudocasuali.

Un generatore di numeri casuali deve produrre sequenze che siano, almeno dal punto di vista statistico, il più possibile indipendenti. Poiché l'algoritmo che produce tali numeri è deterministico, l'obiettivo è costruire procedure che simulino bene il comportamento di una successione indipendente.

Un punto importante è che il grado di "casualità" richiesto dipende fortemente dall'applicazione. In alcune simulazioni fisiche o nei metodi Monte Carlo sono richieste proprietà statistiche molto rigorose.

Un riferimento classico sull'argomento è

> D. E. Knuth, *Seminumerical Algorithms*, volume 2 di *The Art of Computer Programming*, Addison-Wesley (1981).

Considerazioni sui generatori Quando si utilizza un generatore di numeri pseudocasuali è sempre opportuno mantenere un atteggiamento critico nei confronti dei risultati prodotti. In altre parole, bisogna sempre controllare che il generatore non introduca correlazioni indesiderate o artefatti statistici.

Un criterio empirico utile è il seguente: se due generatori diversi producono risultati significativamente differenti per lo stesso esperimento numerico, è probabile che almeno uno dei due presenti problemi.

> *"Be suspicious. Once you've been suspicious, stop and be suspicious again."*
> - Riccardo Mannella

1.0.2 Generatori congruenziali lineari

Uno dei generatori più elementari e diffusi è il *linear congruential generator*. La sequenza viene costruita tramite la relazione ricorsiva:

$$x_{n+1} = ax_n + c \pmod{m}.$$

Qui a è il moltiplicatore, c è l'incremento, m è il modulo. Scelti a, c, m ed un '*seed*' x_n, la sequenza è continua da sé in modo del tutto deterministico.

Questo tipo di generatore ha il vantaggio di essere molto veloce e di poter essere implementato con grande efficienza anche a basso livello.

Tuttavia presenta alcuni limiti importanti. Le sequenze generate possono mostrare correlazioni interne. Innanzitutto perché la successione può assumere solamente valori fra 0 ed $m - 1$

In particolare, se si rappresentano graficamente i punti $(x_n, x_{n+1}, \ldots, x_{n+k-1})$ in uno spazio k-dimensionale, si osserva che tali punti non riempiono uniformemente lo spazio, ma si dispongono su un insieme di iperpiani.

Più precisamente, i punti generati occupano uno spazio di dimensione $k - 1$ e il numero massimo di tali piani è al più $m^{1/k}$, per questo si scelgono sempre alti valori di m per non vedere la periodicità.

Per esempio, scegliendo $m = 2^{32}$, e $k = 3$, si ottiene $m^{1/k} \approx 1600$.

Altri esempi con $k = 3$:

- per $a = 421$, $c = 1663$, e $m = 7875$, 100 punti e $m^{1/k} \approx 19.9$;
- per $a = 4096$, $c = 150889$, e $m = 714025$, 10'000 punti e $m^{1/k} \approx 63.8$.

Questo significa che i punti tridimensionali generati dal metodo si dispongono su un numero relativamente limitato di piani, evidenziando quindi una struttura geometrica non desiderata nella sequenza pseudocasuale.

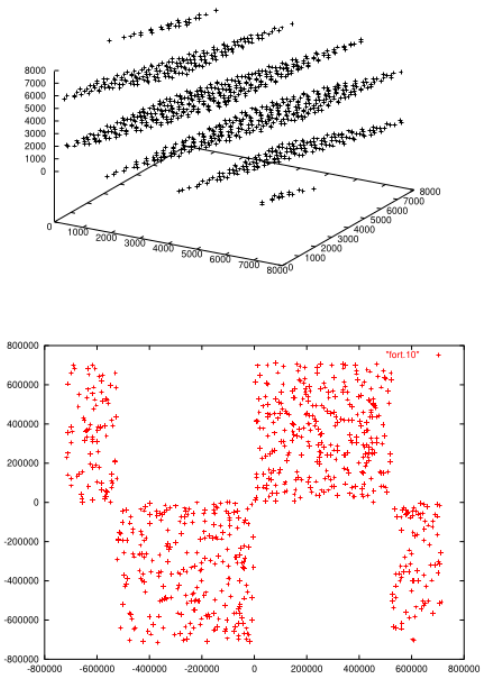


FIG. 1.1: Come si dispongono i punti della successione pseudo-casuale? si nota l'emergere di iperpiani affini che indicano correlazione statistica. La rappresentazione è tridimensionale perché $k = 3$. Gli assi rappresentano i punti $(x_n, x_{n+1}, x_{n+3-1})$.

La rappresentazione grafica mostra chiaramente che i punti non riempiono uniformemente lo spazio tridimensionale, ma si dispongono su un insieme relativamente piccolo di piani.

1.0.3 Combinazione di generatori

Una possibile strategia per migliorare le proprietà statistiche dei generatori congruenziali consiste nel combinare più generatori tra loro.

Un esempio è la routine `RAN1.F`, che combina tre metodi congruenziali differenti. In particolare, vengono utilizzati generatori distinti per i bit più significativi, per quelli meno significativi e per una procedura di *reshuffling* della sequenza.

Se la velocità di esecuzione è il fattore principale, è possibile utilizzare una combinazione più semplice di due generatori congruenziali. Questa strategia è adottata nella routine `RAN2`. Tuttavia, in questo caso il generatore tende a campionare una porzione più limitata dello spazio delle configurazioni.

```fortran
!!! RAN1.F90
! questa è solo una routine di esempio
REAL FUNCTION ran1(IDUM)
    ! Generatore di numeri pseudo-casuali uniformi in (0,1)
    ! Se IDUM < 0 oppure se è la prima chiamata, la routine
    ! inizializza lo stato interno del generatore.
    implicit none

    integer, intent(inout) :: idum
    integer :: j
    integer, save :: iff = 0
    integer, save :: ix1, ix2, ix3
    real, save :: r(97)
    ! Queste variabili devono conservare il loro valore tra chiamate successive alla funzione.
    ! Parametri (m, a, c) dei tre generatori congruenziali:
    integer, parameter :: m1 = 259200, ia1 = 7141, ic1 = 54773
    integer, parameter :: m2 = 134456, ia2 = 8121, ic2 = 28411
    integer, parameter :: m3 = 243000, ia3 = 4561, ic3 = 51349
    real,    parameter :: rm1 = 3.8580247e-6
    real,    parameter :: rm2 = 7.4373773e-6

    IF (idum < 0 .OR. iff = 0) THEN
        iff=1
        ix1=mod(ic1-idum,m1)
        ix1=mod(ia1*ix1+ic1,m1)
        ix2=mod(ix1,m2)
        ix1=mod(ia1*ix1+ic1,m1)
        ix3=mod(ix1,m3)
        DO j=1, 97
            ix1=mod(ia1*ix1+ic1,m1)
            ix2=mod(ia2*ix2+ic2,m2)
            r(j)=(REAL(ix1)+REAL(ix2)*rm2)*rm1
        CONTINUE
        idum=1
    ENDIF

```

```
    ix1=mod(ia1*ix1+ic1,m1)  ! Aggiornamento dei tre generatori congruenziali
    ix2=mod(ia2*ix2+ic2,m2)
    ix3=mod(ia3*ix3+ic3,m3)

    j=1+(97*ix3)/m3 ! Selezione dell'elemento della tabella di shuffle

    IF(j > 97 .OR. j < 1) then
        stop "Errore in ran1: indice fuori intervallo"
    ENDIF
    ! Controllo di sicurezza: j dovrebbe sempre stare tra 1 e 97.

    ran1=r(j)
    ! Il numero pseudo-casuale restituito è l elemento j-esimo della tabella di shuffle.

    ! L elemento appena estratto viene sostituito con un nuovo valore costruito dai due generatori ix1 e ix2.
    ! Questo è il cuore del meccanismo di shuffle.
    r(j)=(REAL(ix1)+REAL(ix2)*rm2)*rm1
    RETURN

END
```

Listing 1.1: RAN1.f90, combinazione di generatori congruenziali, esempio ispirato al *"Numerical Recipes"* di Press, Teukolsky, Vetterling, Flannery. Nota bene: questa è solamente una routine, per essere utilizzata va chiamata

1.0.4 Generatori basati su sistemi caotici

Un approccio alternativo consiste nello sfruttare il comportamento caotico di alcuni sistemi dinamici.

L'idea è quella di utilizzare un sistema dinamico deterministico che presenti una dinamica caotica controllata, e impiegare la successione dei suoi stati come sequenza pseudocasuale.

Un esempio classico è dato dalla mappa logistica:

$$x_{n+1} = \lambda x_n(1 - x_n) \quad . \tag{1.1}$$

Al variare del parametro λ il sistema in eq.(1.1) mostra un comportamento sempre più complesso, fino a entrare in regime caotico (come si vede in figura).

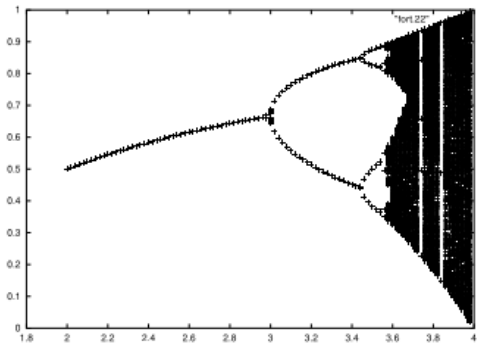


Fig. 1.2: La curva logistica, sembra di essere tornati a lezione da Paolini, wow

Tuttavia l'utilizzo diretto di sistemi caotici come generatori pseudocasuali presenta diversi problemi pratici. In particolare, è necessario stabilire quanto il comportamento caotico del sistema sia effettivamente adeguato per simulare una sequenza statisticamente indipendente.

1.0.5 Generatori come sistemi dinamici a molti stati

Un modo più efficace per costruire generatori pseudocasuali consiste nel progettare sistemi dinamici con un numero elevato di stati interni.

L'idea è quella di costruire una macchina a molti stati in cui il valore successivo della sequenza dipende da una combinazione di stati precedenti. Questo schema può essere rappresentato come una rete di variabili interne che interagiscono tra loro attraverso operazioni di somma o sottrazione.

Questa idea è alla base dei cosiddetti algoritmi *add and carry* oppure *subtract and carry*. Tra i generatori di questo tipo si trovano, ad esempio,

- la routine `RAN3`: si mantiene una tabella di numeri precedenti e si genera il nuovo valore come differenza di due elementi della sequenza (In forma semplificata: $x_n = x_{nj} - x_{nk}$);
- il generatore `rcarry`: questo è un generatore della famiglia subtract-with-carry. È simile a quello precedente ma introduce una variabile aggiuntiva chiamata carry (l'idea è: $x_n = x_{nr} - x_{ns} - c$).

Questi algoritmi presentano in genere periodi di ripetizione molto lunghi e buone proprietà statistiche (risolvendo parzialmente il problema della struttura geometrica in iperpiani).

1.1 Generazione di distribuzioni non uniformi

1.1.1 Dal caso uniforme a una distribuzione arbitraria

Dopo aver costruito un generatore di numeri pseudo–casuali uniformi, il problema successivo consiste nel produrre variabili aleatorie con una distribuzione assegnata. In altre parole, dato un generatore che fornisce variabili uniformi, vogliamo ottenere campioni distribuiti secondo una densità prefissata.

Sia dunque x una variabile casuale distribuita uniformemente in un certo intervallo, con densità $p(x)$, e supponiamo di voler costruire una nuova variabile y con densità $p(y)$. La relazione fondamentale tra le due densità si ottiene imponendo l'invarianza della probabilità sotto trasformazione di variabile. Si ha infatti $|p(y)\,dy| = p(x)\,dx$, da questa relazione segue immediatamente che:

$$p(y) = p(x) \left| \frac{dx}{dy} \right| .$$

Questa formula esprime il fatto che la densità della nuova variabile dipende dalla densità di partenza e dal modo in cui la trasformazione $x \mapsto y$ dilata o contrae gli intervalli.

Il problema centrale diventa allora quello di determinare una trasformazione opportuna tra x e y. Ciò si riconduce a risolvere $\frac{dx}{dy} = f(y)$.

In sostanza, assegnata la distribuzione voluta, si cerca una funzione di trasformazione tale che la variabile uniforme iniziale venga mappata nella nuova variabile con la densità desiderata.

1.1.2 Metodo della funzione cumulativa inversa

Il procedimento precedente si chiarisce riscrivendo la trasformazione in termini di funzione integrale della densità.

Ricordando la relazione $\frac{dx}{dy} = f(y)$ ed integrando rispetto a y, si ottiene

$$x = F(y) \quad ,$$

dove la funzione $F(y)$ è data, a meno di costanti additive o di opportune normalizzazioni, dall'integrale della densità:

$$F(y) = \int p(y)\, dy \quad .$$

Se la funzione F è invertibile, allora si può esprimere y in funzione di x come

$$y(x) = F^{-1}(x).$$

Questo è il contenuto essenziale del metodo della trasformazione inversa: si genera una variabile uniforme x, si costruisce la funzione cumulativa associata alla densità desiderata, e infine si ottiene la variabile distribuita secondo $p(y)$ applicando l'inversa della cumulativa.

Il metodo funziona quindi molto bene quando l'inversa dell'integrale di $p(y)$ è disponibile in forma esplicita, oppure può essere calcolata numericamente in modo efficiente.

Esempio: distribuzione esponenziale

Un esempio particolarmente importante è quello della distribuzione esponenziale. Si consideri la trasformazione

$$y(x) = -\ln x.$$

Se x è uniforme nell'intervallo $(0, 1)$, allora

$$\frac{dx}{dy} = -e^{-y}, \qquad \left|\frac{dx}{dy}\right| = e^{-y}.$$

Poiché la densità di x è uniforme, cioè costante su $(0, 1)$, dalla relazione generale

$$p(y)\, dy = p(x) \left|\frac{dx}{dy}\right| dy$$

segue che, a meno della costante uniforme $p(x)$, la densità di y è

$$p(y)\,dy = e^{-y}\,dy.$$

Si ottiene dunque la distribuzione esponenziale.

Interpretazione geometrica del metodo

Ripartiamo da $F(y) = \int p(y)\,dy$; si osserva che la variabile uniforme x viene ottenuta come valore della cumulativa associata a y. Invertendo questa relazione, si ha ancora

$$y(x) = F^{-1}(x).$$

Geometricamente, la funzione $F(y)$ cresce dal valore minimo al valore massimo della probabilità cumulata. Se si estrae un numero uniforme x tra 0 e 1, lo si può interpretare come un livello di probabilità cumulata; il valore corrispondente di y si ottiene leggendo sull'asse orizzontale il punto in cui la cumulativa raggiunge proprio quel livello.

In altre parole, il metodo della trasformazione inversa trasforma una distribuzione uniforme della probabilità cumulata in una distribuzione non uniforme della variabile fisica.

1.1.3 Trasformazioni in più dimensioni

Il metodo di trasformazione non è limitato al caso unidimensionale, ma si estende anche a distribuzioni in più variabili.

Se $(x_1, x_2, \dots)$ è un vettore di variabili di partenza e $(y_1, y_2, \dots)$ è il nuovo vettore di variabili, allora la relazione tra le densità è regolata dal determinante jacobiano della trasformazione:

$$p(y_1, y_2, \dots)\,dy_1 dy_2 \cdots = \left|\frac{\partial(x_1, x_2, \dots)}{\partial(y_1, y_2, \dots)}\right| p(x_1, x_2, \dots)\,dy_1 dy_2 \cdots .$$

Questa espressione generalizza direttamente la formula unidimensionale.

Anche qui il significato è lo stesso: la densità si modifica in funzione della deformazione locale dei volumi elementari operata dalla trasformazione.

1.1.4 Generazione di distribuzioni gaussiane: metodo di Box-Muller

Un'applicazione molto importante del caso multidimensionale è la generazione di variabili gaussiane: il metodo di Box-Muller, che permette di ottenere due variabili gaussiane indipendenti a partire da due variabili uniformi indipendenti.

Si considerino due variabili uniformi indipendenti x_1 e x_2, entrambe distribuite uniformemente in $(0, 1)$. Si definiscono allora le nuove variabili

$$y_1 \;=\; \sqrt{-2\ln x_1}\,\cos(2\pi x_2) \;=\; r\cos\theta,$$

$$y_2 \;=\; \sqrt{-2\ln x_1}\,\sin(2\pi x_2) \;=\; r\sin\theta.$$

Queste formule costituiscono la trasformazione di Box–Muller. Le due variabili uniformi x_1 e x_2 vengono reinterpretate come variabili ausiliarie che costruiscono, rispettivamente, il raggio e l'angolo in coordinate polari nel piano delle variabili gaussiane. In particolare, $r = \sqrt{-2\ln x_1}$ e $\theta = 2\pi x_2$.

Questa trasformazione è progettata in modo tale che la densità congiunta di (y_1, y_2) risulti gaussiana e radialmente simmetrica, così da avere una routine che trasformi due variabili indipendenti distribuite uniformemente in due variabili distribuite gaussianamente.

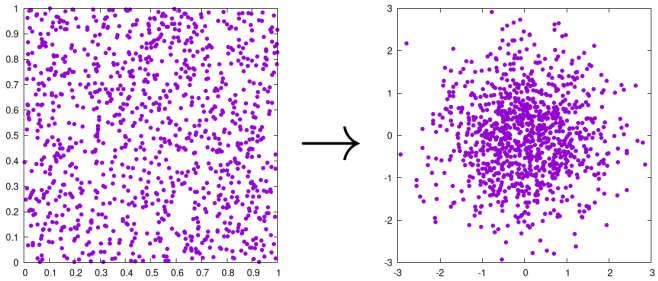


FIG. 1.3: Immagine presa da QUI <–

Calcolo del Jacobiano nella trasformazione di Box-Muller Per giustificare il metodo bisogna analizzare lo jacobiano della trasformazione. Partiamo dalla trasformazione sopra presentata, con definizioni di y_1, y_2, r, θ. Si ricava:

$$x_1 = e^{-r^2/2}, \qquad x_2 = \frac{\theta}{2\pi}.$$

Poiché x_1 e x_2 sono uniformi e indipendenti, la loro densità congiunta è costante nel quadrato unitario:$p(x_1, x_2) = 1, \qquad 0 < x_1 < 1,\ 0 < x_2 < 1.$

Ora, il passaggio da (x_1, x_2) a (r, θ) produce il fattore jacobiano

$$\begin{vmatrix} \dfrac{\partial x_1}{\partial r} & \dfrac{\partial x_1}{\partial \theta} \\ \dfrac{\partial x_2}{\partial r} & \dfrac{\partial x_2}{\partial \theta} \end{vmatrix} = \begin{vmatrix} -re^{-r^2/2} & 0 \\ 0 & \frac{1}{2\pi} \end{vmatrix}.$$

si ottiene $\left|\dfrac{\partial(x_1, x_2)}{\partial(r, \theta)}\right| \;=\; \dfrac{r}{2\pi}\, e^{-r^2/2}$.

Per passare poi da coordinate polari a coordinate cartesiane si usa il noto elemento di area $dy_1\, dy_2 = r\, dr\, d\theta$, cioè $dr\, d\theta = \dfrac{1}{r}\, dy_1\, dy_2$.

Combinando i due passaggi, si trova che la densità di (y_1, y_2) è proporzionale a $\dfrac{r}{2\pi} e^{-r^2/2}\cdot \dfrac{1}{r} \;=\; \dfrac{1}{2\pi} e^{-r^2/2}$. Poiché $r^2 = y_1^2 + y_2^2$, segue che:

$$P(y_1, y_2) = \frac{1}{2\pi} \exp\left(-\frac{y_1^2}{2} - \frac{y_2^2}{2}\right).$$

Questa è precisamente la distribuzione normale bidimensionale standard con componenti indipendenti.

1.1.5 Osservazione conclusiva su questa parte

Riassumendo, il principio generale è il seguente: una volta ottenute variabili uniformi affidabili, si possono costruire distribuzioni non uniformi applicando trasformazioni opportune. Nel caso unidimensionale il punto chiave è l'inversione della cumulativa; nel caso multidimensionale entra in gioco il determinante jacobiano della trasformazione.

Questo quadro teorico prepara direttamente al metodo di *rejection*, che diventa essenziale quando l'inversa della cumulativa non è disponibile in forma trattabile.

1.1.6 Rejection method

Il metodo del rifiuto permette di generare distribuzioni non uniformi senza richiedere il calcolo esplicito dell'integrale della densità.

Si consideri una distribuzione di probabilità desiderata $p(x)$, e una funzione $f(x)$ tale che $f(x) > p(x) \quad \forall x$, l'idea è utilizzare una coppia di variabili casuali uniformi per campionare la distribuzione $p(x)$.

Si procede nel seguente modo:

- si estrae un valore x_s uniformemente distribuito nel dominio;
- si estrae una seconda variabile casuale x_1 uniforme tra 0 e $f(x_s)$;
- si accetta x_s se vale la condizione $x_1 < p(x_s)$

Se la condizione non è soddisfatta, il punto viene rifiutato e si ripete la procedura. Questo metodo non richiede la conoscenza della funzione cumulativa, ma soltanto una funzione $f(x)$ che sovrastimi $p(x)$.

Interpretazione geometrica: Il metodo può essere interpretato geometricamente. Si consideri il grafico delle funzioni $p(x)$ e $f(x)$, con $f(x)$ tale che $f(x) \geq p(x)$

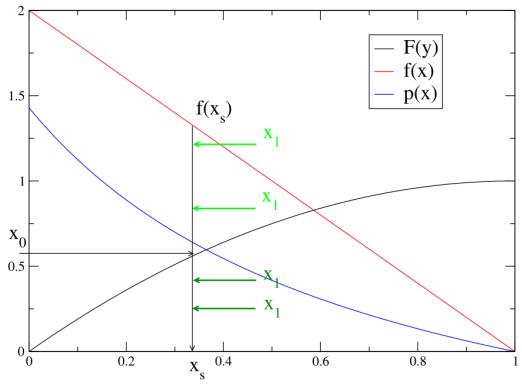


FIG. 1.4: rappresentazione grafica del *rejection method*

Si generano punti uniformemente nella regione delimitata da $f(x)$.

Un punto viene accettato se cade sotto la curva $p(x)$.

In questo modo, i punti accettati risultano distribuiti secondo la densità desiderata.

In fig.(**??**), i punti accettati sono quelli che cadono sotto $p(x)$, mentre gli altri vengono scartati.

1.1.7 Metodo del rifiuto per la distribuzione Gamma

La distribuzione Gamma di ordine intero $a > 0$ rappresenta la distribuzione del tempo di attesa per il verificarsi del a-esimo evento in un processo di Poisson con valore medio unitario ed è data da:

$$p_a(x)\,dx \;=\; \frac{x^{a-1}e^{-x}}{\Gamma(a)}\,dx \quad , \qquad x > 0 \quad , \tag{1.2}$$

Per valori piccoli di a, è possibile generare direttamente la distribuzione come somma di variabili esponenziali indipendenti.

Questo equivale a calcolare il prodotto di a variabili casuali uniformi e successivamente prendere il logaritmo.

Per valori più grandi di a, la distribuzione assume una forma a campana: con massimo circa in $x \approx a$ e larghezza dell'ordine $\sqrt{a}$

Funzione di confronto

Per applicare il metodo del rifiuto, si introduce una funzione di confronto.

Una scelta conveniente che si rivela utile è la distribuzione di Lorentz:

$$p(y)\,dy = \frac{1}{\pi(1+y^2)}\,dy \tag{1.3}$$

Integrandola si ottiene

$$F(y) = \int p(y)\,dy = \frac{1}{\pi}\arctan(y) \tag{1.4}$$

Funzione di maggiorazione

Si introduce una funzione $f(x)$ della forma

$$f(x) \;=\; \frac{c_0}{1 \,+\, (x-x_0)^2/a_0^2} \tag{1.5}$$

Questa funzione deve essere scelta in modo tale che $f(x) \geq p(x)$ e che il prodotto $a_0\,c_0$ sia il più piccolo possibile, al fine di minimizzare il numero di rifiuti.

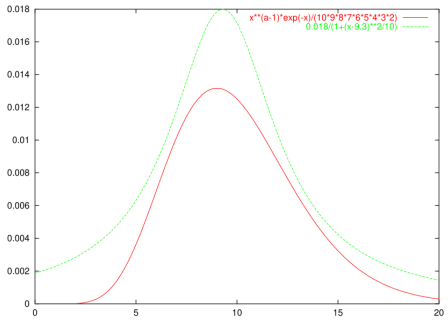


FIG. 1.5: *Gamma function* in rosso, mentre in verde si vede la lorentziana, funzione di maggiorazione $f(x)$.

Algoritmo

Si genera una variabile uniforme $u \in [0,1]$ e si pone $x = a_0 \tan(\pi u) + x_0$ cioè $x = f^{-1}(y)$.

Successivamente si verifica la condizione di accettazione (rejection test).

Questo procedimento è implementato, ad esempio, nella routine `GAMDEV`.

```c
#include <math.h>
float gamdev(int ia, long *idum){
\\ Returns a deviate distributed as a gamma distribution of integer order ia,
\\ ie, a waiting time to the i-th event in a Poisson process of unit mean,
\\ using ran1(idum) as the source of uniform deviates.

    float ran1(long *idum);
    void nrerror(char error_text[]);
    int j;
    float am,e,s,v1,v2,x,y;
    if (ia < 1) nrerror("Error in routine gamdev");
    if (ia < 6) {
    \\ Use direct method, adding waiting
    x = 1.0;
    times.
    for (j=1;j<=ia;j++) x *= ran1(idum);
    x = -log(x);
    }
    else {      \\ Use rejection method.
        do {
            do {
                \\ These four lines generate the tangent of a random angle,
                \\ ie, they are equivalent to y = tan{ * ran1(idum)}.
                do {
                    v1 = ran1(idum);
                    v2 = 2.0*ran1(idum) - 1.0;
                } while (v1*v1+v2*v2 > 1.0);
                y= v2 / v1;
                am = ia - 1;
                s = sqrt( 2.0*am+1.0 );
                x = s*y + am;
            } while (x <= 0.0);
            e=(1.0+y*y)*exp(am*log(x/am)-s*y);
        } while (ran1(idum) > e);
    }
return x;

\\ We decide whether to reject x: Reject in region of zero probability.
\\ Ratio of prob. fn. to comparison fn. Reject on basis of a second uniform deviate.
}
```

Listing 1.2: Implementazione del metodo GAMDEV

1.2 Integrazione Monte Carlo

Il metodo di integrazione Monte Carlo è strettamente collegato al metodo del rifiuto.

Esso permette di stimare integrali multidimensionali mediante campionamento casuale.

In uno spazio n-dimensionale, dato un insieme di N punti x_i, vale la stima

$$\int_V f\, dV \;\approx\; V\langle f\rangle \pm V\sqrt{\frac{\langle f^2\rangle - \langle f\rangle^2}{N}} \tag{1.6}$$

dove

$$\langle f^n\rangle = \frac{1}{N}\sum_{i=1}^{N} f^n(x_i) \tag{1.7}$$

Interpretazione

L'idea è quella di incorporare una regione difficile da campionare all'interno di una regione più semplice, dalla quale è possibile estrarre punti uniformemente.

Successivamente si utilizza il campionamento casuale per stimare l'integrale.

Il metodo può essere scritto in molti modi diversi in base a in quante dimensioni stiamo campionando, in quale spazio estraiamo i nostri punti e che forma ha la funzione, ma il concetto rimane sempre questo.

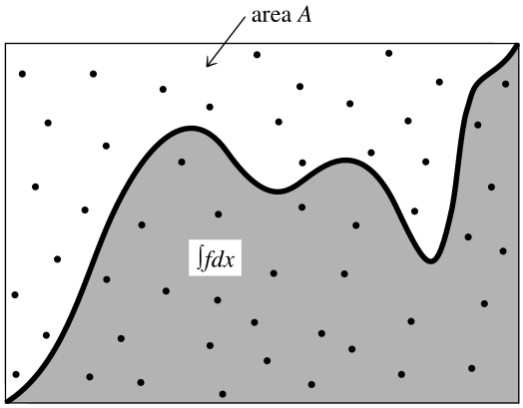


FIG. 1.6: Esempio visuale preso da "*Numerical Recipes in C*". L'integrale della funzione f è stimato come l'area del rettangolo A moltiplicata per la frazione di punti che si trova sotto la curva f

Esempio: area di un cerchio

Si consideri il calcolo dell'area di un cerchio unitario.

Si definisce una funzione

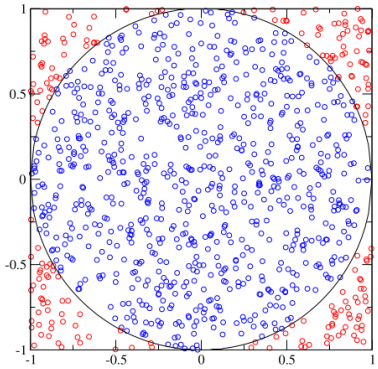
$$f = \begin{cases} 1 & \text{all'interno del cerchio} \\ 0 & \text{all'esterno} \end{cases} \tag{1.8}$$

Il volume della regione di campionamento è V = 4, la stima dell'area è $\int_V f\, dV \approx V\langle f\rangle$

Per $N = 1000$ si ottiene

$$\begin{aligned} \langle f\rangle &= 0.797 \\ \text{area} &= 3.19 \pm 0.05 \end{aligned}$$

Per $N = 10^6$:



$$\begin{aligned} \langle f \rangle &= 0.78616 \\ \text{area} &= 3.145 \pm 0.005 \end{aligned}$$

FIG. 1.7: Integrale Monte Carlo dell'area di un cerchio

Esempio: disco con foro

Si consideri un disco di raggio 1 con un foro di raggio 0.5 centrato nel punto $(0.5, 0)$.

Si vogliono calcolare:

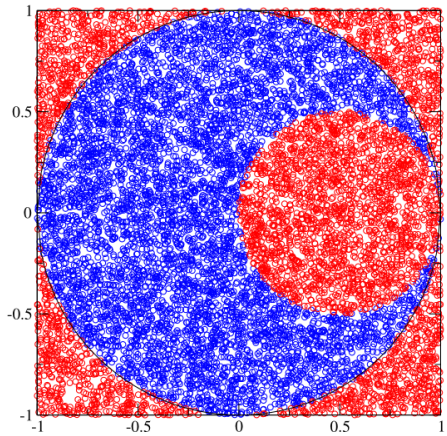
$$x_{cm} = \frac{\int x \, dV}{\int dV} \tag{1.9}$$

$$I_{0,0} = \int (x^2 + y^2) \, dV \tag{1.10}$$

Le funzioni da integrare sono:

- $f_x = x$ all'interno della regione, zero fuori;
- $f_y^2 = y^2$, ecc.

Per $N = 10^6$ si ottiene:



$$\begin{aligned} x_{cm} &= -0.393 \ \pm \ 0.004 \\ y_{cm} &= -0.000 \ \pm \ 0.005 \\ I_{0,0} &\approx 1.275 \end{aligned}$$

FIG. 1.8: Integrale Monte Carlo dell'area di un cerchio forato

```python
import numpy as np
import matplotlib.pyplot as plt


def inside_holed_disk(x, y):
    """
    Restituisce True se (x,y) appartiene al disco grande
    ma non al foro interno.
```

```
    """
    in_big_disk = x**2 + y**2 <= 1.0**2
    in_hole = (x - 0.5)**2 + y**2 <= 0.5**2
    return in_big_disk & (~in_hole)


def monte_carlo_holed_disk(n_samples=100_000, seed=1234):
    """
    Campiona punti uniformi nel quadrato [-1,1]x[-1,1]
    e stima:
      - area della regione
      - integrale di x
      - integrale di y
      - integrale di x^2+y^2
      - centro di massa
      - momento d'inerzia rispetto all'origine
    """
    rng = np.random.default_rng(seed)

    # Campionamento uniforme nel quadrato
    x = rng.uniform(-1.0, 1.0, n_samples)
    y = rng.uniform(-1.0, 1.0, n_samples)

    # Area del quadrato contenitore
    V = 4.0

    # Maschera della regione
    mask = inside_holed_disk(x, y)

    # Funzioni da integrare, prolungate con zero fuori dalla
        regione
    f_area = mask.astype(float)
    f_x = np.where(mask, x, 0.0)
    f_y = np.where(mask, y, 0.0)
    f_inertia = np.where(mask, x**2 + y**2, 0.0)

    # Stime Monte Carlo degli integrali
    area_est = V * np.mean(f_area)
    int_x_est = V * np.mean(f_x)
    int_y_est = V * np.mean(f_y)
    inertia_est = V * np.mean(f_inertia)

    # Errori statistici (deviazione standard della media)
    area_err = V * np.std(f_area, ddof=1) / np.sqrt(n_samples)
    int_x_err = V * np.std(f_x, ddof=1) / np.sqrt(n_samples)
    int_y_err = V * np.std(f_y, ddof=1) / np.sqrt(n_samples)
    inertia_err = V * np.std(f_inertia, ddof=1) / np.sqrt(
        n_samples)

    # Centro di massa: x_cm = ( x dA)/(Area), y_cm = ( y dA)/(
        Area)
```

```
    x_cm_est = int_x_est / area_est
    y_cm_est = int_y_est / area_est

    results = {
        "x": x,
        "y": y,
        "mask": mask,
        "area": area_est,
        "area_err": area_err,
        "int_x": int_x_est,
        "int_x_err": int_x_err,
        "int_y": int_y_est,
        "int_y_err": int_y_err,
        "x_cm": x_cm_est,
        "y_cm": y_cm_est,
        "inertia": inertia_est,
        "inertia_err": inertia_err,
    }

    return results


def plot_sample_points(x, y, mask, max_points=15000):
    """
    Mostra i punti Monte Carlo:
      - blu = punti accettati nella regione
      - grigio chiaro = punti fuori
    """
    n = min(len(x), max_points)

    x_plot = x[:n]
    y_plot = y[:n]
    mask_plot = mask[:n]

    fig, ax = plt.subplots(figsize=(7, 7))

    ax.scatter(x_plot[~mask_plot], y_plot[~mask_plot], s=4,
       alpha=0.3, label="Fuori regione")
    ax.scatter(x_plot[mask_plot], y_plot[mask_plot], s=4,
       alpha=0.6, label="Dentro regione")

    # Disegno del bordo del disco esterno
    t = np.linspace(0, 2*np.pi, 500)
    ax.plot(np.cos(t), np.sin(t), linewidth=2)

    # Disegno del bordo del foro
    ax.plot(0.5 + 0.5*np.cos(t), 0.5*np.sin(t), linewidth=2)

    ax.set_aspect("equal")
    ax.set_xlim(-1.05, 1.05)
    ax.set_ylim(-1.05, 1.05)
```

```
    ax.set_title("Campionamento Monte Carlo del disco forato")
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.show()


def running_area_estimate(n_samples=50_000, seed=1234):
    """
    Calcola la stima progressiva dell'area per visualizzare la convergenza.
    """
    rng = np.random.default_rng(seed)
    x = rng.uniform(-1.0, 1.0, n_samples)
    y = rng.uniform(-1.0, 1.0, n_samples)

    mask = inside_holed_disk(x, y).astype(float)

    cumulative_mean = np.cumsum(mask) / np.arange(1, n_samples + 1)
    area_running = 4.0 * cumulative_mean

    return area_running


def plot_running_area(area_running):
    """
    Mostra la convergenza della stima dell'area.
    """
    fig, ax = plt.subplots(figsize=(8, 4.5))
    ax.plot(area_running, linewidth=1.2, label="Stima Monte Carlo progressiva")

    # Valore esatto dell'area:
    # area disco grande - area foro = pi - pi/4 = 3pi/4
    exact_area = 3.0 * np.pi / 4.0
    ax.axhline(exact_area, linestyle="--", linewidth=2, label="Valore esatto 3/4")

    ax.set_title("Convergenza della stima dell'area")
    ax.set_xlabel("Numero di campioni")
    ax.set_ylabel("Area stimata")
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.show()


if __name__ == "__main__":
    results = monte_carlo_holed_disk(n_samples=200_000, seed=42)
```

```python

    print("=== Stime Monte Carlo ===")
    print(f"Area          = {results['area']:.6f} ś {results['area_err']:.6f}")
    print(f" x dA        = {results['int_x']:.6f} ś {results['int_x_err']:.6f}")
    print(f" y dA        = {results['int_y']:.6f} ś {results['int_y_err']:.6f}")
    print(f"x_cm          = {results['x_cm']:.6f}")
    print(f"y_cm          = {results['y_cm']:.6f}")
    print(f"I_O           = {results['inertia']:.6f} ś {results['inertia_err']:.6f}")

    plot_sample_points(results["x"], results["y"], results["mask"], max_points=12000)

    area_running = running_area_estimate(n_samples=50_000, seed=42)
    plot_running_area(area_running)
```

Listing 1.3: Esercizio di integrazione Monte Carlo del disco bucato in Python, scritta da ChatGpt 5.3

Capitolo 2

Ricerca degli zeri di una funzione

2.1 Introduzione

La determinazione degli zeri di una funzione, ovvero la risoluzione dell'equazione $f(x) = 0$, costituisce uno dei problemi fondamentali dell'analisi numerica. Nonostante la formulazione apparentemente semplice, si tratta in generale di un problema non banale quando non si ha una soluzione analitica.

L'idea di base è la seguente: si cerca inizialmente di individuare in maniera approssimata una regione dello spazio in cui è presente uno zero della funzione; successivamente si utilizza un algoritmo numerico per raffinare progressivamente la stima.

Esempio introduttivo: Consideriamo la funzione

$$f(x) = x + \frac{x^2}{10},$$

e supponiamo di studiarla nell'intervallo

$$[-15,\ 10].$$

Un primo approccio consiste nel valutare la funzione in un numero finito di punti (ad esempio 30 punti equispaziati) e osservare il comportamento del grafico.

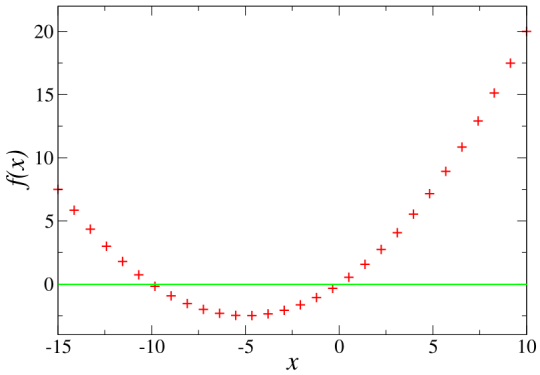


FIG. 2.1: esempio di funzione $f(x)$ plottata con soli 30 punti per stimare i segmenti in cui cercare lo zero.

Questo permette di individuare visivamente la regione in cui la funzione cambia segno, e quindi dove è presente uno zero.

Questa fase preliminare è essenziale: senza una buona stima iniziale della posizione dello zero, molti metodi numerici risultano inefficaci o addirittura divergenti.

Casi difficili e comportamenti patologici: Esistono tuttavia funzioni per le quali la localizzazione dello zero può risultare estremamente complicata dal punto di vista numerico.

Un esempio significativo è dato dalla funzione

$$f(x) = 3x^2 + 1 + \frac{\ln\left[(\pi - x)^2\right]}{\pi^4}.$$

In questo caso esistono due zeri situati approssimativamente in

$$x \approx \pi \pm 10^{-667}.$$

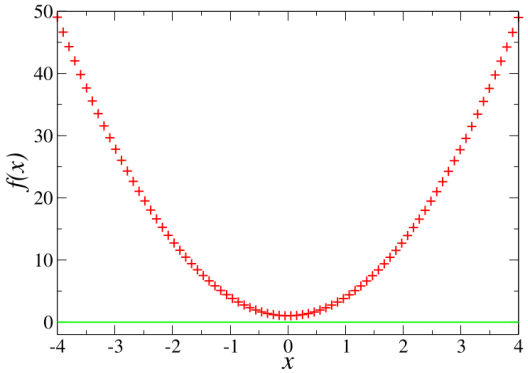


FIG. 2.2: Funzione $f(x)$ plottata con 80 punti nel range [-4; 4]

Si tratta di zeri estremamente vicini al valore π, tanto che una loro individuazione diretta tramite semplice campionamento risulta praticamente impossibile.

Tuttavia, osservando la struttura della funzione, si nota che il termine logaritmico diventa molto grande e negativo quando x si avvicina a π. Questo suggerisce che lo zero si trovi nelle vicinanze di tale punto.

Questo esempio mostra chiaramente come, in molti casi, sia necessario combinare osservazioni qualitative sulla funzione con tecniche numeriche.

Nel seguito assumeremo di essere riusciti a individuare un intervallo che contiene uno zero della funzione e ci concentreremo su come raffinarne la posizione.

2.2 Metodo di bisezione

Una volta individuato un intervallo in cui è presente uno zero, il metodo più semplice per determinarlo con precisione è il **metodo di bisezione**.

Questo metodo si basa su una proprietà fondamentale delle funzioni continue: se una funzione cambia segno in un intervallo, allora deve necessariamente annullarsi in almeno un punto interno.

Idea del metodo Supponiamo di conoscere due punti x_1 e x_2 tali che la funzione cambia segno nell'intervallo $[x_1, x_2]$, ovvero:

$$f(x_1) > 0, \qquad f(x_2) < 0,$$

Si definisce quindi il punto medio $x_m = \frac{x_1+x_2}{2}$ e si valuta il segno della funzione nello stesso:

- se $f(x_m) > 0$, lo zero si trova nell'intervallo $[x_m, x_2]$, quindi si pone $x_1 = x_m$;
- se $f(x_m) < 0$, lo zero si trova nell'intervallo $[x_1, x_m]$, quindi si pone $x_2 = x_m$.

Il procedimento viene quindi iterato, restringendo progressivamente l'intervallo che contiene lo zero.

Stima dell'errore: Se inizialmente la lunghezza dell'intervallo è $\epsilon = |x_1 - x_2|$, dopo n iterazioni la lunghezza dell'intervallo si riduce a $\delta \approx \epsilon / 2^n$.

Questo fornisce una stima diretta dell'errore sulla posizione dello zero.

Esempio numerico del metodo di bisezione: Consideriamo la funzione

$$f(x) = \frac{x^3}{5} + \frac{x}{5} + 0.1.$$

Applicando il metodo di bisezione si ottiene la seguente sequenza di approssimazioni:

i	x_r	δ
1	-0.9	0.9
2	-0.45	0.45
3	-0.23	0.23
4	-0.338	0.11
5	-0.394	0.06
6	-0.422	0.03

Si osserva come l'intervallo contenente lo zero si restringa progressivamente e la stima della radice converga.

Questo esempio illustra chiaramente il comportamento del metodo: la convergenza è garantita, ma relativamente lenta, poiché l'errore si riduce esponenzialmente con fattore 1/2 ad ogni iterazione.

2.2.1 Osservazioni sul metodo di bisezione

Il metodo di bisezione è estremamente robusto e presenta alcune caratteristiche fondamentali, esso converge purché la funzione sia continua e si riesca a individuare un intervallo in cui essa cambia segno. Tuttavia, la convergenza è relativamente lenta, in quanto l'errore si riduce secondo la legge $\delta = \epsilon/2^n$.

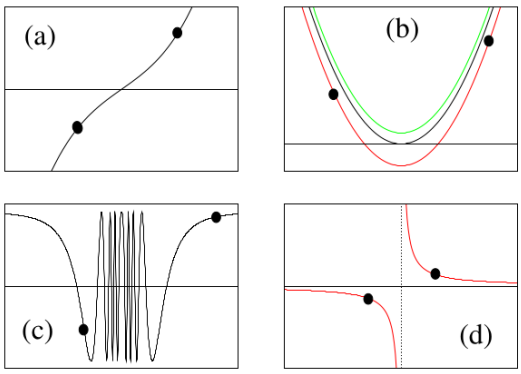


FIG. 2.3: Casi particolari che si possono verificare.

Esistono diversi casi particolari che è opportuno considerare:

- **Caso normale**: la funzione cambia segno una sola volta nell'intervallo e il metodo converge allo zero corretto.
- **Radici multiple**: la funzione può toccare l'asse senza cambiare segno; in questo caso il metodo può non individuare correttamente lo zero.
- **Radici molto vicine**: possono esistere più zeri estremamente vicini tra

loro, rendendo difficile l'identificazione numerica.

- **Discontinuità**: se la funzione presenta una discontinuità, il metodo converge comunque, ma al punto di discontinuità anziché a uno zero reale.

2.3 Metodo di Newton-Raphson

Un approccio più sofisticato per il raffinamento della radice è il metodo di **Newton-Raphson**: supponiamo di avere una stima iniziale x della radice e di voler trovare una stima migliorata x_r. Si approssima la funzione tramite lo sviluppo di Taylor al primo ordine:

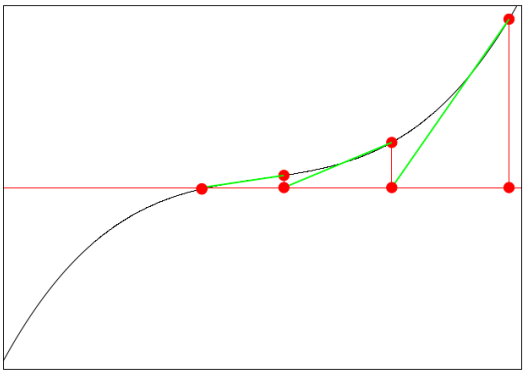
$$f(x_r) \approx f(x) + (x_r - x)\frac{df(x)}{dx} \stackrel{!}{=} 0 \quad .$$

Imponendo la condizione $f(x_r) \stackrel{!}{=} 0$, da cui segue immediatamente

$$x_r = x - \frac{f(x)}{f'(x)}. \quad \Rightarrow \quad x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad . \tag{2.1}$$

Questa relazione definisce una procedura iterativa.

Esempio numerico: Consideriamo nuovamente la funzione $f(x) = \frac{x^3}{5} + \frac{x}{5} + 0.1$.



Applicando il metodo di Newton-Raphson si ottiene la seguente sequenza:

i	x_r	$f(x_r)$
0	1.9	
1	1.1173	0.60
2	0.4825	0.22
4	-0.4714	0.02
5	-0.4257	0.0006
7	-0.4239	$< 10^{-8}$

Si osserva una convergenza molto più rapida rispetto al metodo di bisezione.

2.3.1 Analisi dell'errore

Per comprendere meglio la velocità di convergenza, consideriamo una radice x_r e poniamo $x = x_r + \epsilon$, così che sviluppando la funzione:

$$f(x+\epsilon) = f(x) + \epsilon f'(x) + \frac{\epsilon^2}{2} f''(x).$$

Applicando l'iterazione di Newton di eq.(2.1) si ottiene per l'errore:

$$\epsilon_{i+1} = -\frac{\epsilon_i^2 f''(x_i)}{2f'(x_i)}.$$

Questo mostra che l'errore si riduce **quadraticamente**, ovvero

$$\epsilon_{i+1} \sim \epsilon_i^2,$$

il che spiega la rapidissima convergenza del metodo.

2.3.2 Osservazioni sul metodo di Newton

Il metodo di Newton-Raphson presenta numerosi vantaggi:

- convergenza molto rapida;
- esatto per funzioni lineari;
- estendibile a più dimensioni;
- non richiede il bracketing della radice.

Tuttavia presenta anche alcune criticità:

- richiede una buona stima iniziale;
- può divergere o oscillare;
- può bloccarsi in punti stazionari;
- fallisce in presenza di discontinuità;
- richiede che $f'(x) \neq 0$.

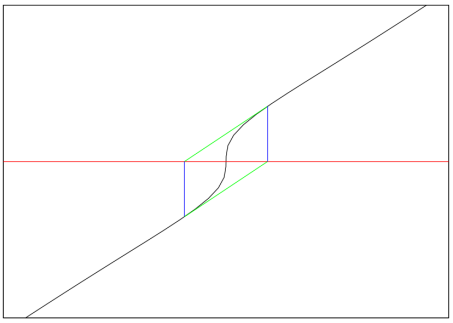


Fig. 2.4: idk

2.4 Metodi per polinomi

Nel caso particolare dei polinomi esistono metodi specifici molto efficienti.

Consideriamo un polinomio

$$P_n(x) \;=\; (x - x_1)\,(x - x_2)\,\cdots\,(x - x_n).$$

Si può scrivere

$$\ln|P_n(x)| \;=\; \sum_i \ln|x - x_i|.$$

Derivando:

$$\frac{d}{dx}\ln|P_n(x)| \;=\; \sum_i \frac{1}{x-x_i} \;=\; \frac{P_n'(x)}{P_n(x)} \;=\; G,$$

$$\frac{d^2}{dx^2}\ln|P_n(x)| \;=\; \sum_i \frac{1}{(x-x_i)^2} \;=\; \left(\frac{P_n'}{P_n}\right)^2 - \frac{P_n''}{P_n} \;=\; H.$$

Si ottiene quindi $G = \frac{1}{a} + \frac{n-1}{b}$ e $H = \frac{1}{a^2} + \frac{n-1}{b^2}$:

$$a = \frac{n}{G \pm \sqrt{(n-1)(nH - G^2)}}.$$

Il metodo consiste nell'iterare $x \to x - a$,scegliendo il segno che massimizza il denominatore.

2.5 Generalizzazione a più dimensioni

Il problema della ricerca degli zeri si estende naturalmente a più dimensioni.

Si considerano sistemi del tipo $f_i(\mathbf{X}) = 0$, dove $\mathbf{X}$ è un vettore in $\mathbb{R}^N$.

Si generalizza il metodo di Newton-Cotes scrivendo

$$f_i(\mathbf{X}+\delta) \;\approx\; f_i(\mathbf{X}) + \sum_j \frac{\partial f_i}{\partial x_j}\,\delta_j.$$

Definendo $a_{ij} = \frac{\partial f_i}{\partial x_j}$, si ottiene il sistema lineare:

$$\sum_j a_{ij}\delta_j = -f_i(\mathbf{X}).$$

Risolvendo per δ si ottiene $\delta_j = -\sum_i (a_{ij})^{-1} f_i(\mathbf{X})$ e si itera seguendo $X_i^{(\text{new})} = X_i^{(\text{old})} + \delta_i$.

Esempio bidimensionale: Un esempio di sistema non lineare è

$$f_1(x,y) = x^2 + y^2 - 1,$$

$$f_2(x,y) = y - x^2 + \frac{x}{2}.$$

La soluzione corrisponde all'intersezione delle due curve.

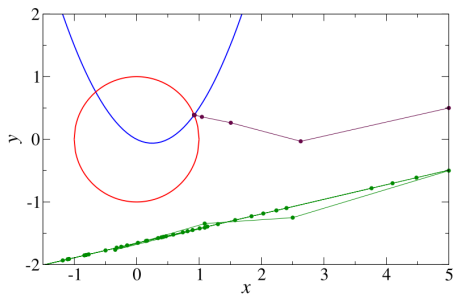


FIG. 2.5: esempio bidimensionale

2.6 Conclusioni

Riassumendo:

- è sempre necessario avere un'idea preliminare della posizione della radice;
- il metodo di bisezione è robusto ma lento;
- il metodo di Newton è rapido ma delicato;
- per i polinomi esistono metodi dedicati;
- in più dimensioni il metodo di Newton è lo strumento principale, ma è sensibile alle condizioni iniziali.

La scelta del metodo più adatto dipende quindi dal problema specifico, dal grado di informazione disponibile sulla funzione e dal compromesso desiderato tra stabilità e velocità di convergenza.

Capitolo 3

Analisi numerica nell'integrazione

3.1 Introduzione

Il problema dell'integrazione numerica può essere risolto, in linea di principio, con l'aiuto di equazioni differenziali ordinarie (ODEs):

$$\int_a^x f(y)\,dy \;=\; g(x) \qquad \Rightarrow \qquad \frac{d\,g}{d\,x} \;=\; f(x)\ ,$$

con boundary $x_0 = a$.

Sebbene questa osservazione sia formalmente corretta, è preferibile sviluppare metodi diretti di integrazione numerica, analogamente a quanto si è fatto per le equazioni differenziali, dove la scelta del metodo appropriato non è affatto immediata.

Un algoritmo generale di integrazione numerica ha la struttura

$$\int_a^b f(x)\,dx \;=\; \sum_i w_i\, f(x_i),$$

dove w_i sono pesi e x_i i nodi di valutazione.

In genere si pone $x_i = a + ih$ con passo h uniforme e si scrive $f_i = f(x_i)$.

Si distinguono due grandi classi di formule:

- **Formule chiuse**: utilizzano anche i valori agli estremi a e b.
- **Formule aperte**: non utilizzano uno o entrambi gli estremi.

Il vantaggio principale delle formule aperte è che possono essere utilizzate quando la funzione non è definita agli estremi, pur essendo l'integrale ben definito.

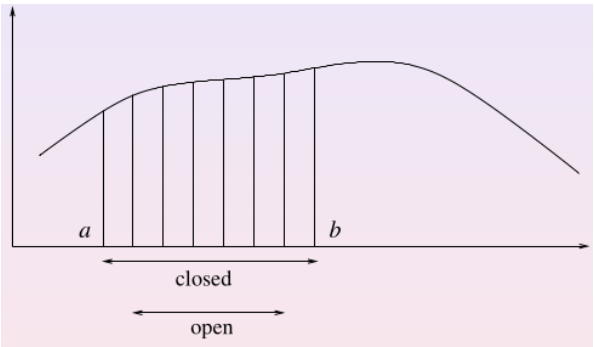


Fig. 3.1: differenza fra intervalli presi in considerazione da classi di formule chiuse e aperte

3.1.1 Formule di NewtonCotes su intervallo singolo

Le formule di Newton-Cotes nascono dalla divisione in intervalli equispaziati e dalla successiva integrazione esatta di un polinomio che passa per quegli n punti (interpolazione).

La regola fondamentale dell'integrazione numerica, la formula di Newton-Cotes di grado più basso è la regola dei trapezi, che usa una retta tesa fra due punti successivi per interpolare:

$$\int_{x_1}^{x_2} f(x)\,dx \;=\; \frac{1}{2}\,h\,[f_1 + f_2] + \mathcal{O}(h^3\,f'') \;, \tag{3.1}$$

questa è esatta per funzioni lineari, ad esempio $f(x) = x$.

Si ottiene interpolando $f(x)$ con il polinomio lineare che passa per i punti $(x_1,\, f_1)$ e $(x_2,\, f_2)$:

$$f(x) \;=\; f_1 \,+\, (f_2 \,-\, f_1)\,x \quad ,$$

e in seguito integrando sull'intervallo normalizzato $0 \le x \le 1$.

Formule di ordine superiore si ottengono interpolando con polinomi di grado più elevato. Ad esempio,

$$\int_{x_1}^{x_3} f(x)\,dx \;=\; h\,\left[\frac{1}{3}\,f_1 \,+\, \frac{4}{3}\,f_2 \,+\, \frac{1}{3}\,f_3\right] \,+\, \mathcal{O}(h^5\,f^{(4)}) \quad , \tag{3.2}$$

che è la regola di Simpson, esatta per polinomi fino al terzo grado.

La regola di Simpson nasce dall'interpolazione con una parabola $P(x) = Ax^x + Bx + C$ su tre punti equispaziati x_0, x_1, x_2 con funzione da integrare $f_0 = f(x_0)$, f_1, f_2. Facciamo un cambio di variabile $t = x - x_i$ così che i tre punti diventino $t = 0$, h, $2h$, riscrivendo la parabola come $P(t) = at^2 + bt + c$.

Si va ora ad imporre che $P(t = 0) = c = f_0$, mentre nel secondo punto $P(t = h) = ah^2 + b^h + c = f_1$ e nel terzo $P(t = 2h) = 4ah^2 + 2bh + c = f_2$. Invertendo questo sistema di equazioni si trova che $c = f_0$, $a = \frac{f_2 - 2f_1 + f_0}{2h^2}$ mentre $b = \frac{f_1 - f_0}{h} - ah$.

A questo punto l'integrale $\int_{x_0}^{x_2} f(x)dx \approx \int_0^{2h} P(t)dt = \int_0^{2h} at^2 + bt + f_0 dt$ è facilmente sostituibile con le dovute sostituzioni di a e b nelle forme appena trovate e con una semplice integrazione del polinomio si ottiene proprio Simpson: $\int_{x_0}^{x_2} f(x)dx \approx \frac{h}{3}[f_0 + 4\,f_1 + f_2]$.

Analogamente, per cinque punti la formula di Newton-Cotes diventa:

$$\int_{x_1}^{x_5} f(x)\,dx \;=\; h\,\left[\frac{14}{45}\,f_1 \,+\, \frac{64}{45}\,f_2 \,+\, \frac{24}{45}\,f_3 \,+\, \frac{64}{45}\,f_4 \,+\, \frac{14}{45}\,f_5\right] \,+\, \mathcal{O}(h^7\,f^{(6)}) \quad .$$

3.1.2 Alcune formule locali

È utile osservare anche relazioni locali su intervalli elementari:

$$\int_{x_0}^{x_1} f(x)\,dx \;=\; h[f_1] \,+\, \mathcal{O}(h^2\,f') \quad ,$$

$$\int_{x_0}^{x_1} f(x)\,dx \;=\; h\left[\frac{3}{2}f_1 \,-\, \frac{1}{2}f_2\right] \,+\, \mathcal{O}(h^3\,f'') \quad .$$

La prima è un semplice rettangolo, una Newton-Cotes aperta di ordine 0 con errore di ordine h^2, mentre la seconda formula è una Newton-Cotes aperta di ordine 1 e si ottiene interpolando linearmente $f(x) \;=\; f_1 + x\,(f_2 - f_1)$, e integrando nell'intervallo $-1 \leq x \leq 0$:

$$\int_{-1}^{0} f(x)\,dx \;=\; -(-f_1) \,-\, \frac{(-1)^2}{2}\,(f_2 - f_1) \;=\; \left[\frac{3}{2}f_1 - \frac{1}{2}f_2\right] \quad .$$

Formule di ordine superiore si ottengono usando approssimazioni polinomiali di grado crescente.

3.1.3 Intervalli estesi

Passando a intervalli estesi, dalla regola dei trapezi in eq.(3.1) segue una regola dei trapezi estesa:

$$\int_{x_1}^{x_n} f(x)\,dx \;=\; h\left[\frac{1}{2}f_1 \,+\, f_2 \,+\, f_3 \,+\, \ldots \,+\, \frac{1}{2}f_n\right] \,+\, \mathcal{O}\left(\frac{(x_n - x_1)^3}{N^2}\,f''\right) \quad , \tag{3.3}$$

dove N è il numero di suddivisioni.

Dalla regola di Simpson in eq.(3.2), si ottiene invece:

$$\int_{x_1}^{x_n} f(x)\,dx \;=\; h\left[\frac{1}{3}f_1 \,+\, \frac{4}{3}f_2 \,+\, \frac{2}{3}f_3 \,+\, \frac{4}{3}f_4 \ldots \,+\, \frac{4}{3}f_{n-1} \,+\, \frac{1}{3}f_n\right] \,+\, \mathcal{O}\left(\frac{1}{N^4}\right) \quad . \tag{3.4}$$

Di seguito un'implementazione in C della regola dei trapezi e di Simpson:

```
1 #include <stdio.h>
2
3 // Queste le funzioni che andremo a definire a seguito del
    MAIN
4 double POWER(double numero,double elevazione); // la funzione
    da integrare: legge di potenza
5 double funzione(double varF); // routine che presa una x
    restituisce la y=f(x)
6 double areaTrapezio(double lato1,double lato2,double altezza);
     // routine che calcola l'area di un singolo trapezio
7 double trapezioIntegral(double a,double b,int colonne); //
    integrazione con metodo dei trapezi
```

```
8  double simpsonIntegral(double a,double b,int colonne); //
       integrazione con Simpson
9
10 int main(int argc,char**argv){
11     // main: qui si chiama TrapezioIntegral per fare un'
           integrazione da a=1 a b=2
12     // con l'intervallo diviso in 5000 parti uguali
13     printf("[ trapezi: %lf ]\n", trapezioIntegral(1,2,5000));
14     printf("[ simpson  %lf ]\n", simpsonIntegral(1,2,5000));
15     }
16
17 double POWER(double numero,int elevazione){
18     double ret = 1;
19     for(int i=elevazione;i>0;i--){
20         ret *= numero;
21         }
22     return ret;
23     }
24 double funzione(double varF){
25     double y = POWER(varF,2);
26     return y;
27     }
28
29 double areaTrapezio(double lato1,double lato2,double altezza){
30     return (lato1+lato2)*altezza/2;
31     }
32
33 double trapezioIntegral(double a,double b,int colonne){
34     double area = 0;
35     double nPartiUguali = (b-a)/colonne;
36
37     for(int i=0;i<colonne-1;i++){
38         area += areaTrapezio(funzione(a),funzione(a+
               nPartiUguali),nPartiUguali);
39         a += nPartiUguali;
40         }
41         area += areaTrapezio(funzione(a),funzione(b),b-a);
42         return area;
43     }
44
45 // Simpson composito su [a,b] con "colonne" sottointervalli (
       deve essere pari)
46 double simpsonIntegral(double a,double b,int colonne){
47     if(colonne < 2) colonne = 2;
48     if(colonne % 2 != 0) colonne -= 1;   // Simpson richiede
           un numero pari di sottointervalli
49
50     double h = (b-a)/colonne;
51     double somma = funzione(a) + funzione(b);
52
53     // termini con coefficiente 4: i dispari
```

```
    for(int i=1; i<=colonne-1; i+=2){
        double x = a + i*h;
        somma += 4.0 * funzione(x);
    }

    // termini con coefficiente 2: i pari (esclusi estremi)
    for(int i=2; i<=colonne-2; i+=2){
        double x = a + i*h;
        somma += 2.0 * funzione(x);
    }

    return (h/3.0) * somma;
}
```

Listing 3.1: Implementazione del metodo dei trapezi

Si può combinare una formula aperta locale (singolo intervallo) con una formula chiusa estesa di quelle appena scritte per ottenere una formula aperta su intervallo esteso.

Ad esempio, combinando

$$\int_{x_0}^{x_1} f(x)\,dx = hf_1 + \mathcal{O}(h^2 f')$$

con la regola dei trapezi estesa su $[x_1, x_n]$ vista in eq.3.3, si ottiene:

$$\int_{x_0}^{x_n} f(x)\,dx \;=\; h\left[\frac{3}{2}f_1 + f_2 + f_3 + \ldots + \frac{3}{2}f_{n-1}\right] + \mathcal{O}\left(\frac{1}{N^2}\right) \quad .$$

Analogamente, combinando la formula aperta di ordine superiore

$$\int_{x_0}^{x_1} f(x)\,dx \;=\; h\left[\frac{23}{12}f_1 - \frac{16}{12}f_2 + \frac{5}{12}f_3\right] + \mathcal{O}(h^4 f^{(3)}) \quad ,$$

con la regola di Simpson estesa eq.(3.4), si ottiene una formula complessiva di ordine $\mathcal{O}(1/N^4)$.

$$\int_{x_0}^{x_n} f(x)\,dx \;=\; h\left[\frac{27}{12}f_1 + 0 + \frac{13}{12}f_3 + \frac{4}{3}f_4 + \ldots + \frac{4}{3}f_{n-4}, + \frac{13}{12}f_{n-3} + 0 + \frac{27}{12}f_{n-1}\right] + \mathcal{O}\left(\frac{1}{N^2}\right) \quad .$$

Il problema pratico fondamentale rimane: quanti passi bisogna utilizzare?

Se si usa la regola dei trapezi come blocco base, l'errore scala come $\boxed{K/N^2}$, raddoppiando N, l'errore si riduce di un fattore 4. Questo comportamento è alla base dei metodi di raffinamento successivi.

3.1.4 Valutazione dellerrore e formula esatta dei trapezi

L'errore può essere valutato facendo la differenza fra l'integrale calcolato con N steps e quello calcolato con 2N steps.

Per stimare in modo più preciso lerrore della regola dei trapezi estesa, si può utilizzare la sua espressione esatta in termini dei numeri di Bernoulli:

$$\int_{x_1}^{x_n} f(x)\,dx \;=\; h\left[\frac{1}{2}f_1 + f_2 + \ldots + \frac{1}{2}f_n\right] - \frac{B_2 h^2}{2!}\left(f_n' - f_1'\right) - \frac{B_{2k}h^{2k}}{(2k)!}\left(f_n^{(2k-1)} - f_1^{(2k-1)}\right) - \ldots$$

dove B_k sono i numeri di Bernoulli, definiti tramite

$$\frac{t}{e^t - 1} \;=\; \sum_{n=0}^{\infty} B_n \frac{t^n}{n!}.$$

Il termine dominante dellerrore è proporzionale a h^2, cioè a $1/N^2$. Se indichiamo con S_N lintegrale calcolato con N suddivisioni e con S_{2N} quello con $2N$, possiamo eliminare il termine in h^2 combinando linearmente i due risultati:

$$S = \frac{4}{3}S_{2N} - \frac{1}{3}S_N.$$

In questo modo lerrore dominante diventa dellordine h^4.

3.1.5 Integrazione di Romberg

Lidea precedente può essere iterata. Se una combinazione elimina il termine in h^2, si può ripetere il procedimento per eliminare anche il termine in h^4, poi quello in h^6, e così via.

Il metodo di Romberg consiste nel:

- calcolare lintegrale con una sequenza di suddivisioni $N, 2N, 4N, \ldots$;
- costruire combinazioni lineari che eliminano progressivamente i termini dominanti dellerrore;
- interpretare il risultato come unestrapolazione razionale nel limite $N \to \infty$.

Il metodo produce una tabella triangolare di approssimazioni con ordine crescente di accuratezza.

Di seguito un'implementazione in C dell'integrazione di Romberg presa dal *Numerical Recipes in C: The art of scientific programming*:

```c
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

#define EPS  1.0e-6f   // accuratezza relativa desiderata
```

```c
#define JMAX 20          // massimo numero di raffinamenti (livelli trapezio)
#define K    5           // punti usati nellestrapolazione (Romberg "ordine" ~ 2K)

// Prototipi: definiscili da qualche parte nel tuo progetto
float trapzd(float (*func)(float), float a, float b, int j);
void  polint(const float xa[], const float ya[], int n, float x, float *y, float *dy);

static void nrerror(const char *msg) {
    fprintf(stderr, "%s\n", msg);
    exit(EXIT_FAILURE);
}

// Romberg via: trapezi successivi + estrapolazione polinomiale in h^2 verso 0
float qromb(float (*func)(float), float a, float b)
{
    float s[JMAX];       // s[j] = stima trapezoidale al livello j (0-based)
    float h[JMAX];       // h[j] = variabile proporzionale a (passo)^2
    float ss = 0.0f;     // stima extrapolata dellintegrale (valore a h^2=0)
    float dss = 0.0f;    // stima errore dellestrapolazione

    // h[0] è solo una scala: conta il rapporto tra livelli.
    // Ad ogni raffinamento, il passo h_geom dimezza => (passo)^2 /4.
    h[0] = 1.0f;

    for (int j = 0; j < JMAX; ++j) {

        // trapezio al livello (j+1) per restare compatibili con NR
        s[j] = trapzd(func, a, b, j + 1);

        // quando ho almeno K valori, estrapolo in h^2 fino a 0
        if (j + 1 >= K) {
            // uso gli ultimi K punti: h[j-K+1 .. j] e s[j-K+1 .. j]
            polint(&h[j - K + 1], &s[j - K + 1], K, 0.0f, &ss, &dss);

            // criterio di arresto: errore stimato relativo sotto EPS
            if (fabsf(dss) <= EPS * fabsf(ss)) {
                return ss;
            }
```

```
        }

        // preparo la "x" per il prossimo livello: h^2 scala
           di 1/4 quando il passo scala di 1/2
        if (j + 1 < JMAX) {
            h[j + 1] = 0.25f * h[j];
        }
    }

    nrerror("Too many steps in routine qromb");
    return 0.0f; // non ci arriva mai
}
```

Listing 3.2: Implementazione del metodo di Romberg

3.1.6 Quadrature Gaussiane

Vogliamo approssimare integrali del tipo

$$\int W(x) f(x)\, dx \;=\; \sum_{j=1}^{N} w_j f(x_j) \quad ,$$

dove: $W(x) > 0$ è una funzione peso, $\int W(x)\, dx = 1$ (normalizzazione) ed i nodi x_j non sono necessariamente equispaziati.

Introduciamo il prodotto scalare pesato

$$\langle f \,|\, g \rangle \;=\; \int W(x)\, f(x)\, g(x)\, dx \quad .$$

Si definisce una successione di polinomi ortogonali $\{p_k(x)\}$ tramite la ricorrenza a tre termini:

$$p_{-1}(x) = 0, \qquad p_0(x) = 1 \quad ,$$

$$p_{j+1}(x) \;=\; (x - a_j)\, p_j(x) - b_j\, p_{j-1}(x) \quad ,$$

con coefficienti:

$$a_j = \frac{\langle x\, p_j \,|\, p_j \rangle}{\langle p_j \,|\, p_j \rangle}, \qquad b_j = \frac{\langle p_j \,|\, p_j \rangle}{\langle p_{j-1} \,|\, p_{j-1} \rangle} \quad .$$

Proprietà fondamentali:

- p_j è monico e di grado j,
- p_j possiede j zeri reali semplici,

- gli zeri sono interlacciati con quelli di p_{j-1}.

Assumiamo che $f(x)$ sia un polinomio di grado al massimo $2N-1$. Può essere scritto in generale come $f(x) = p_N(x)q_f(x) + r_f(x)$ con $q_f(x)$ e $r_f(x)$ polinomi di grado $< N$. In particolare:

$$q_f(x) = \sum_{N-1} a_k p_k(x) \ , \qquad a_k = \frac{\langle p_k | q_f \rangle}{\langle p_k | p_k \rangle} \ ,$$

$$r_f(x) = \sum_{N-1} b_k p_k(x) \ , \qquad b_k = \frac{\langle p_k | r_f \rangle}{\langle p_k | p_k \rangle} \ ,$$

Vale che:

$$\int W(x)\, f(x)\, dx \ = \ \int W(x)\, (P_N(x) q_f(x) + r_f(x))\, dx \ = \ \sum w_j\, (P_N(x) q_f(x) + r_f(x)) \ ,$$

che, <u>nel caso in cui le x_j siano radici di p_N</u>, diventa:

$$\int W(x)\, f(x)\, dx \ = \ \sum w_j(x)\, r_f(x_j) \ = \ \int W(x)\, r_f(x)\, dx \ ,$$

Ma per ipotesi, r_f era di un grado in meno rispetto ad N, quindi l'integrale qui sopra è esatto.

Sappiamo che le x_j sono radici di p_n, cosa dire per le w_j?

Grazie all'ortogonalità si ottiene il sistema lineare:

$$\begin{pmatrix} p_0(x_1) & p_0(x_2) & \dots & p_0(x_N) \\ p_1(x_1) & p_1(x_2) & \dots & p_1(x_N) \\ \vdots & \vdots & & \vdots \\ p_{N-1}(x_1) & p_{N-1}(x_2) & \dots & p_{N-1}(x_N) \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{pmatrix} = \begin{pmatrix} \int W(x) p_0(x)\, dx \\ 0 \\ \vdots \\ 0 \end{pmatrix} \ .$$

Se scegliamo x_j come zeri di $p_N(x)$, il sistema è ben posto e la formula risulta esatta fino al grado $2N-1$.

I pesi si possono esprimere direttamente come:

$$w_j = \frac{\langle p_{N-1} | p_{N-1} \rangle}{p_{N-1}(x_j)\, p'_N(x_j)} .$$

Intervallo	$W(x)$	"*Recurrence*"	Nome
$(-1,1)$	1	$(i+1)P_{i+1} \ = \ (2i+1)xP_i \ + \ iP_{i-1}$	**Legendre**
$(-1,1)$	$\dfrac{1}{\sqrt{1-x^2}}$	$T_{i+1} \ = \ 2xT_i \ - \ T_{i-1}$	**Chebyshev**
$(0,\infty)$	$x^c e^{-x}$	$(i+1)L^c_{i+1} \ = \ (-x+2i+c+1)L^c_i$ $-(i+c)L^c_{i-1} \quad c = 0,1,\dots$	**Laguerre**
$(-\infty,\infty)$	e^{-x^2}	$H_{i+1} \ = \ 2xH_i \ - \ 2iH_{i-1}$	**Hermite**

Tabella GaussLegendre

Per $W(x) = 1$ su $(-1, 1)$:

n	x_j	w_j
2	± 0.577350269189626	1.0
3	± 0.774596669241483	0.555555555555556
	0	0.888888888888889
4	± 0.861136311594053	0.347854645137454
	± 0.339981043584856	0.652145154862546

Per integrare su (a, b) mappiamo l'intervallo in (-1,1), grazie al cambio di variabile:

$$y = \frac{2(x-a)}{b-a} - 1 \quad , \qquad x = a + \frac{(y+1)(b-a)}{2} \quad .$$

Allora si ottiene:

$$\int_a^b f(x)\,dx \;=\; \frac{b-a}{2}\int_{-1}^{1} f\left(a + \frac{(y+1)(b-a)}{2}\right) dy.$$

3.1.7 Integrali impropri

Un integrale è <u>improprio</u> se:

- uno dei limiti è infinito,
- la funzione diverge in un estremo ma è integrabile,
- è presente una singolarità interna.

Limiti infiniti

Trattando un integrale improprio con limiti infiniti (che può voler dire anche solo molto molto grandi), per $ab > 0$ si può fare un cambio di variabile del tipo:

$$\int_a^b f(x)\,dx \;=\; \int_{1/a}^{1/b} \frac{1}{t^2} f\left(\frac{1}{t}\right) dt \quad ,$$

Chiaro che un cambio di variabile del genere significa che rimaniamo a debita distanza dal punto $x = 0$. Se $ab < 0$, si divide lintegrale in due parti in modo che l'integrale che contiene lo zero non sia trasformato.

Singolarità integrabile allestremo

Supponiamo che l'integrale abbia una singolarità integrabile di tipo *power law*:

$$f(x) \sim (x-a)^{-\gamma} \;, \qquad 0 \le \gamma < 1 \quad ,$$

si usa la mappatura $t = (x-a)^{1-\gamma}$ così che si ottenga:

$$\int_a^b f(x)\,dx \;=\; \frac{1}{1-\gamma}\int_0^{(b-a)^{1-\gamma}} t^{\frac{\gamma}{1-\gamma}}\, f\Big(t^{\frac{1}{1-\gamma}} + a\Big)\,dt \quad ,$$

con $b > a$. Similmente se la singolarità è in b:

$$\int_a^b f(x)\,dx \;=\; \frac{1}{1-\gamma}\int_0^{(b-a)^{1-\gamma}} t^{\frac{\gamma}{1-\gamma}}\, f\Big(b - t^{\frac{1}{1-\gamma}}\Big)\,dt \quad ,$$

Alternativamente un metodo più banale ma più situazionale è aggiungere e sottrarre una funzione con stessa singolarità di cui si conosce l'integrale.

Singolarità interna

Se la divergenza è in un punto interno $x = y$ vi sono due casi:

- L'integrando è tale che la divergenza è integrabile su (a, y) e (y, b). In tal caso si divide lintervallo e si utilizza la prescrizione per integrale improprio con limite infinito;
- Se lintegrale esiste solo come valore principale, si riorganizza lintegrando per ottenere cancellazione numerica.

Singolarità di tipo Cauchy

Un integrale che andiamo spesso a risolvere in fisica è nella forma:

$$I(y) \;=\; \mathrm{P}\int_a^b \frac{f(x)}{x-y}\,dx \quad ,$$

Uno dei modi di risolvere quest'integrale, che assume che $f(x)$ sia continua nella sua derivata prima e seconda, usa l'integrazione per parti:

$$I(y) \;=\; \left[\ln|x-y|\,f(x)\right]_a^b - \int_a^b \ln|x-y|\,f'(x)\,dx \quad .$$

Se necessario, si integra nuovamente:

$$\begin{aligned} I(y) \;=\; & \left[\ln|x-y|\,f(x)\right]_a^b - \left[(x-y)\ln|x-y| - (x-y)\right] f'(x)\Big|_a^b + \ldots \\ & \ldots + \int_a^b \left[(x-y)\ln|x-y| - (x-y)\right] f''(x)\,dx \quad . \end{aligned}$$

Ora l'integrando finale è regolare salvo discontinuità di f''.

Formule aperte

Una formula aperta utile è:

$$\begin{aligned}\int_{x_1}^{x_n} f(x)\,dx \;&= h\left[f_{3/2} + f_{5/2} + \cdots + f_{n-1/2}\right] + \ldots \\ &\ldots + \sum_{k=1}^{\infty} \frac{B_{2k} h^{2k}}{(2k)!}\left(1 - 2^{-2k+1}\right)\left(f_n^{(2k-1)} - f_1^{(2k-1)}\right) \quad .\end{aligned}$$

Qui i B_{2k} sono numeri di Bernoulli.

In questo caso la semplice riduzione del passo non permette il riuso diretto dei punti precedenti; tuttavia triplicando la suddivisione si possono costruire procedure di raffinamento analoghe a Romberg.

Capitolo 4

Integrazione numerica delle Equazioni Differenziali Ordinarie (ODE)

Il problema che andiamo a studiare in questa parte è la soluzione numerica di un problema del tipo $\boxed{y' = f(x, y)}$ con condizione iniziale $y(x_0) = y_0$ in un dominio $x_0 \leq x \leq x_f$, ridurremo anche ogni equazione differenziale ordinaria (da ora in avanti ODE) di ordine superiore ad un sistema di ODE di primo ordine:

$$y'' = f(x, y, y') \quad \Rightarrow \begin{cases} y' = v \\ v' = f(x, y, v) \end{cases} .$$

Data una ODE del tipo $y'' = f(x, y, y')$, abbiamo due diversi problemi:

1. **Initial value problems** (IVPs): $y(x_0)$ e $y'(x_0)$ sono dati ed y e y' per x e x_f vanno trovati;

2. **Two-point value problems** (TVPs): le condizioni ai bordi sono date per più di una x, ad esempio conosciamo $y(x_0)$ e $y'(x_0)$ e vogliamo trovare y e y' per ogni x.

Per risolvere numericamente un'ODE si introduce una discretizzazione dell'asse x, costruendo una mesh del tipo $x_0 < x_1 < \cdots < x_i < \cdots < x_f$. L'integrazione consiste nell'avanzare la soluzione da x_i a x_{i+1} utilizzando un passo h adeguato. È quindi necessario un algoritmo che:

1. Dato $y(x_i)$, fornisca una stima di $y(x_{i+1})$. Questo costituisce il **metodo di integrazione**;

2. Uno **stepper** che adatta il passo, valuta l'errore locale associato al metodo di integrazione e decide se mantenere, aumentare o ridurre il passo h per garantire stabilità ed efficienza;

3. Un codice che gestisce l'inizio e la fine dell'integrazione fra x_0 e x_f.

4.0.1 Metodi multi-step

Data la ODE $y' = f(x, y)$ e un *time mesh* $\{x_0, x_1, \ldots, x_n\}$, con $x_i = ih$, un metodo multistep di ordine p esprime y_{n+1} come combinazione lineare di alcuni valori già calcolati $y_n, y_{n-1}, \ldots$ e delle corrispondenti derivate $y'_n, y'_{n-1}, \ldots$ moltiplicate per lo step h (vedi metodi lineari multistep):

$$\boxed{y_{n+1} = \sum_{j=0}^{p} a_j y_{n-j} + h \sum_{j=-1}^{p} b_j y'_{n-j}} \tag{4.1}$$

Stiamo facendo un'espansione *'Taylor-like'*, come utilizzarla? Iniziamo derivando un semplice schema esplicito al secondo ordine **Adams' type** ($\equiv a_j = 0 \ \ \forall \ \ j > 1$):

$$y_{n+1} \ = \ a_0 y_n \ + \ h \left[b_0 y'_n + b_1 y'_{n-1} \right] \tag{4.2}$$

Scegliamo una funzione polinomiale di prova, $y_{n+j} = (t + jh)^3$ e la sostituiamo in eq.4.2 ottenendo l'identità:

$$(t + h)^3 = a_0 t^3 + h[3b_0 t^2 + 3b_1 (t - h)^2] \qquad ,$$

espandendo e matchando i coefficienti fra membro sinistro e destro si trovano: $a_0 = 1$, $b_1 = -1/2$, $b_0 = 3/2$ ed un errore $\propto 6h^3/2$ dato che non compare un termine h^3 nel membro destro.

Metodi impliciti

I metodi impliciti sono quelli per cui $b_{-1} \neq 0$:

$$y_{n+1} = \sum_{j=0}^{p} a_j y_{n-j} + h \sum_{j=-1}^{p} b_j y'_{n-j} \tag{4.3}$$

Quindi, mentre in un metodo esplicito non comparirebbe a destra un termine con y'_{n+1}, nei metodi impliciti compare. Normalmente, si fa un guess del nuovo y_{n+1} via un metodo esplicito (in cui $b_{-1} = 0$, quindi calcoliamo y_{n+1} tramite valori conosciuti) per poi fare un certo numero di iterazioni utilizzando un metodo implicito.

Famiglie di metodi multistep di tipo Adams

Abbiamo definito gli integratori Adams' type come quelli per cui $a_j = 0 \ \ \forall \ \ j > 0$ di seguito in tabella si vedono i coefficienti per metodi di due diverse famiglie derivate da questa prima ipotesi:

- **Adams-Bashforth** (AB(0) meglio noto come **metodo di Eulero**, AB(1), ..., AB(4), ...), questi sono dei metodi espliciti che assumono la forma:

$$\boxed{y_{n+1} = y_n + h \sum_{j=0}^{p} b_j \, y'_{n-j}} \tag{4.4}$$

- Mentre similmente possiamo includere il termine b_{-1} per avere una forma di tipo Adams ma implicita, detta di **Adams-Moulton** (AM(-1), AM(0) o meglio noto come **metodo dei trapezi**, ..., AM(3), ...):

$$\boxed{y_{n+1} = y_n + h \sum_{j=-1}^{p} b_j \, y'_{n-j}} \tag{4.5}$$

ordine p	b_0	b_1	b_2	b_3	b_4
0	1				
1	3/2	$-1/2$			
2	23/12	$-16/12$	5/12		
3	55/24	$-59/24$	37/24	$-9/24$	
4	1901/720	$-2774/720$	2626/720	$-1274/720$	251/720

Tab. 4.1: Adams-Bashforth

ordine p	b_{-1}	b_0	b_1	b_2	b_3
-1	1				
0	1/2	1/2			
1	5/12	8/12	$-1/12$		
2	9/24	19/24	$-5/24$	1/24	
3	251/720	646/720	$-264/720$	106/720	$-19/720$

Tab. 4.2: Adams-Moulton

analisi dell'ordine di errore

L'idea è imporre che lo schema riproduca esattamente l'andamento di una funzione polinomiale di grado noto, in modo da determinare i coefficienti che compaiono nella combinazione lineare. Il risultato finale è un metodo che utilizza y'_n e y'_{n-1} per approssimare y_{n+1}, con un errore globale dell'ordine di h^3.

Per la funzione $y' = 2x$, con condizione iniziale $y(0) = 0$, la soluzione esatta $y = x^2$ ma risolvendola con uno schema $y_{n+1} = y_n + 2hx_n$ vediamo che ridurre il passo di integrazione h non porta ad una precisione arbitrariamente alta. Esiste quindi un valore ottimale del passo h che minimizza l'errore totale come si vede in fig.4.1. Infatti, l'errore totale è composto da:

- l'errore di troncamento, che decresce all'aumentare della risoluzione,
- l'errore numerico di roundoff, che invece cresce se h è troppo piccolo, poiché aumenta il numero totale di operazioni.

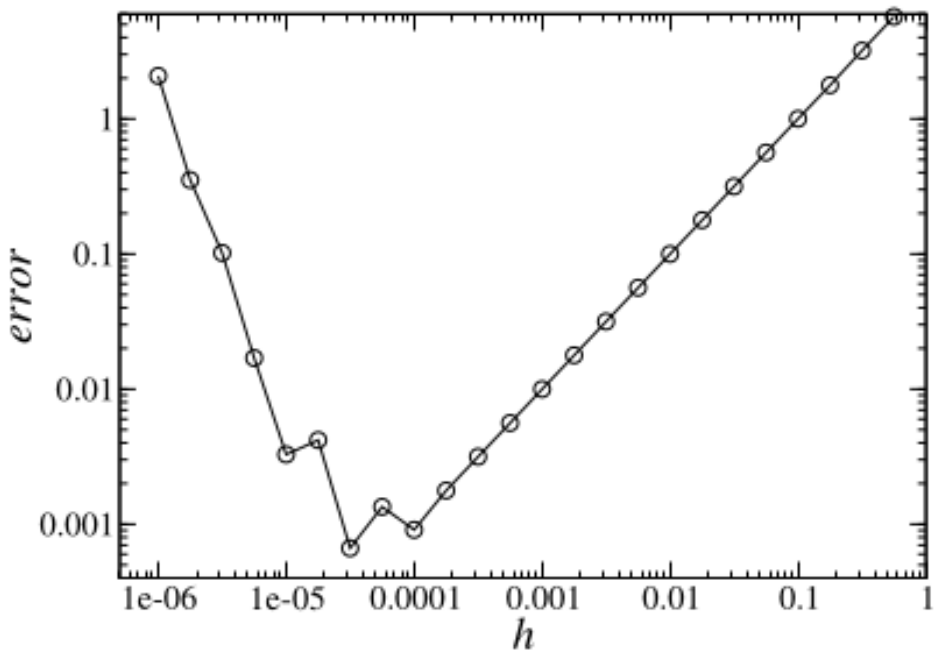


FIG. 4.1: errore di un metodo di Eulero $y_{n+1} = y_n + 2hx_n$ in funzione di h, usato per integrare la funzione $y' = 2x$ con condizione iniziale $y(0) = 0$

stabilità, radici caratteristiche, regioni di A-stabilità

Sia $\{y_i\}$ la soluzione numerica corrispondente ai punti della mesh $x_0, x_1, \ldots, x_n$, con passo h. Il metodo è *stabile* se per ogni perturbazione iniziale $|z_i - y_i| < \varepsilon$ per $i < p$, la differenza rimane limitata:

$$|z_i - y_i| < C\varepsilon \qquad \text{per ogni } i \geq p.$$

Il metodo **converge** se y_i è tale che $Max_{i<p}|Y_i - y_i| \to 0$ se $h \to 0$ e vale anche $Max_{i>p}|Y_i - y_i| \to 0$, allora y_i si dice convergere a Y_i.

Un metodo **consistente** ($\sum_{j=0}^{p} a_j = 1$) *è convergente se e solo se è stabile.*

Il problema della stabilità è difficile, notiamo però che in generale integrare è un problema legato al trovare le radici di un polinomio, esprimiamo ancora il punto y_{n+1} come combinazione lineare dei punti vicini e le derivate:

$$y_{n+1} = \sum_{j=0}^{p} a_j y_{n-j} + h \sum_{j=-1}^{p} b_j y'_{n-j} \quad ,$$

dove in generale vale $\sum_{j=0}^{p} a_j = 1$ mentre $\sum_{j=0}^{p} j a_j = \sum_{j=-1}^{p} b_j$

La stabilità e la convergenza sono assicurate dalla *root condition*:

$$\rho(r) = r^{p+1} - \sum_{j=0}^{p} a_j r^{p-j}$$

questo polinomio ha $p+1$ radici, notiamo inotre che $\rho(1) = 0$ cioè 1 è sempre una radice. La condizione ci dice che:

- $|r_j| \leq 1$ per $j = 0, 1, \ldots, p$
- $|r_j| = 1$ se r_j è una radice semplice

e - se soddisfatto - implica che il metodo sia stabile. Richiamiamo il generico *multi-step* di eq.4.1, in questo caso $p = 1$ e $a_0 = 1$, le radici di $0 = r^2 - r$ sono 0 ed 1 quindi il metodo è stabile.

Si ottengono condizioni più stringenti se si considera un'equazione differenziale test $y' = \lambda y(x)$, che ha soluzione di tipo se ci chiediamo le radici in questo caso abbiamo:

$$\rho^{n+1} = \sum_{j=0}^{p} a_j \rho_{n-j} + \lambda h \sum_{j=-1}^{p} b_j \rho_{n-j} \quad ,$$

da cui ricaveremmo $p+1$ radici $\rho_i(\lambda h)$:

- Il metodo è **relativamente stabile** se $|\rho_j(\lambda h)| \leq \rho_0(\lambda h)$ per $j = 1, \ldots, p$;
- Il metodo è **debolmente stabile** altrimenti.

Per il nostro multi-step : $a_0 = 1$, $b_0 = 3/2$, $b_1 = -1/2$, $p = 1$:

$$\rho^2 - (1 + 3\lambda h/2)\rho + \lambda h/2 = 0 \qquad \Rightarrow \qquad \rho = \frac{1 + 3\lambda h/2 \pm \sqrt{(1 + 3\lambda h/2)^2 - 2\lambda h}}{2}$$

$$\Rightarrow \qquad \rho_{0,1} \sim 1 + \lambda h, \quad \frac{\lambda h}{2} \qquad \text{il metodo è relativamente stabile, ma } \lambda < 0$$

Invece il metodo è **assolutamente stabile** se $|\rho_j(\lambda h)| < 1$ per $j = 0, 1, \ldots, p$. Nel caso del nostro multi-step abbiamo scritto sopra i valori e stavolta facciamo una considerazione senza approssimazione su

$$\rho_{0,1} = \frac{1 + 3\lambda h/2 \pm \sqrt{(1 + 3\lambda h/2)^2 - 2\lambda h}}{2} \quad ,$$

che è A-stabile per $-1 < \lambda h < 0$

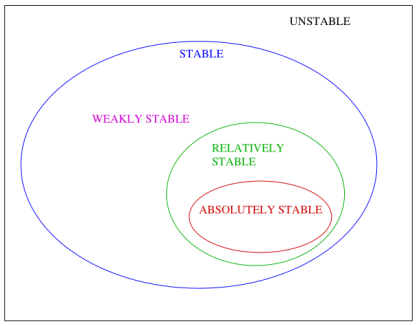


FIG. 4.2: Schema visivo per comprendere i diversi tipi di stabilità

Per un metodo numerico, la soluzione approssimata soddisfa

$$y_{n+1} = \rho(z)\, y_n, \qquad z = h\lambda,$$

dove $\rho(z)$ è il fattore di amplificazione numerico. Una regione di stabilità è il luogo geometrico dei valori complessi $z = h\lambda$ per cui il metodo numerico NON amplifica errori quando la soluzione vera è smorzante, la *regione di stabilità assoluta* è l'insieme dei valori $z \in \mathbb{C}$ tali che $|\rho(z)| < 1$.

La figura mostra un esempio regioni di stabilità:

- in rosso: i metodi Adams–Bashforth (AB), espliciti;
- in verde: i metodi Adams–Moulton (AM), impliciti.

Si osserva che i metodi AB hanno regioni di stabilità più piccole, mentre i metodi AM sono significativamente più stabili.

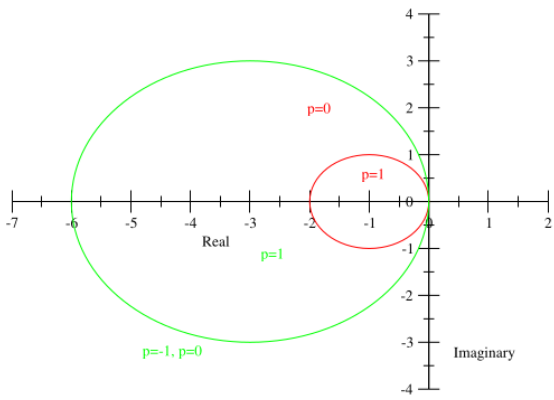


FIG. 4.3: Esempio di una regione di stabilità

Confronto di metodi multi-step

Guardiamo un primo confronto fra alcuni metodi che possiamo scrivere viste le cose affrontate:

- **metodo di Eulero**: $y_{n+1} = y_n + hy'_n + O(h^2)$, relativamente stabile;
- metodo del punto medio (**midpoint**): $y_{n+1} = y_{n-1} + 2hy'_n + O(h^3)$, debolmente stabile;
- Adams-Bashforth (**AB(2)**): $y_{n+1} = y_n + h(3y'_n - y'_{n-1})/2 + O(h^3)$, assolutamente stabile;
- **metodo dei trapezi**: $y_{n+1} = y_n + h(y'_n - y'_{n+1})/2 + O(h^3)$, stabile, per $\lambda < 0$ è assolutamente stabile.

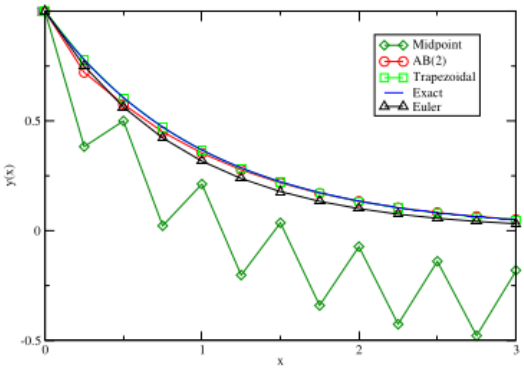


FIG. 4.4: Integrazione di $y' = -y$ con h=0.25 e condizioni al contorno $y(0) = 1$

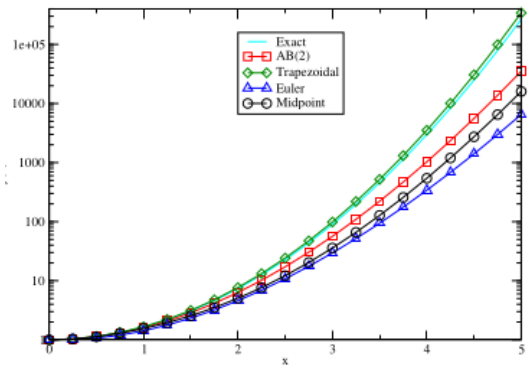


FIG. 4.5: Integrazione di $y' = xy$ con h=0.25 e condizioni al contorno $y(0) = 1$

Eulero che è relativamente stabile e di ordine h^2 è da considerarsi migliore del midpoint che sarà anche di ordine h^3 ma è solamente debolmente stabile. Però, in un sistema già instabile - ad esempio $y' = xy$ in cui la soluzione è un esponenziale - conta invece solo l'ordine di errore.

4.0.2 Metodi one-step

Nei metodi one-step, y_{n+1} dipende unicamente da y_n, si capisce bene come il vantaggio più grande sia non dover tenere in memoria le y_{n-j} che usavamo nei metodi multi-step.

Il metodo one-step più banale è una comune espansione di Taylor della soluzione:

$$y(t+h) = y(t) + hy'(t) + \frac{h^2}{2}y''(t) + \cdots ,$$

fermandosi alla derivata n-esima si ottiene un metodo numerico di ordine desiderato. Questo metodo non è generalmente utilizzato ma è molto facile da usare e appropriato quando la soluzione è di tipo polinomiale.

Poiché $y' = f(t, y(t))$, la seconda derivata si ricava tramite la regola della catena:

$$y'' \;=\; \frac{\partial}{\partial t}f(t,y) + \frac{\partial}{\partial y}f(t,y)\frac{\partial}{\partial y}y(t) \;=\; f_t(t,y) + f_y(t,y)f(t,y).$$

Un metodo di Taylor del secondo ordine è quindi dato da:

$$y_{n+1} = y_n + h\, f(t_n, y_n) + \frac{h^2}{2}\left[f_t(t_n, y_n) + f_y(t_n, y_n)f(t_n, y_n)\right].$$

In molti casi, metodi come il trapezoidale implicito o i metodi di Runge–Kutta permettono di raggiungere lo stesso ordine di accuratezza con un numero minore di derivate richieste.

metodi Runge-Kutta

L'idea dei metodi Runge-Kutta (RK) è quella di approssimare l'integrale

$$y(t+h) = y(t) + \int_t^{t+h} f(\tau, y(\tau))\, d\tau,$$

costruendo una stima accurata della pendenza media della soluzione nel tratto $[t, t+h]$. A differenza dei metodi multistep, i metodi RK utilizzano soltanto le informazioni al punto (t, y) corrente, senza ricorrere ai valori precedenti. Per comprendere come si arriva a tali schemi, possiamo ricostruire passo per passo la procedura intuitiva che porta dapprima a un metodo di ordine due (RK2) e poi al classico metodo di ordine quattro (RK4).

Prendiamo ad esempio la funzione $y(x)$ soluzione dell'ODE $y' = f(x,\, y(x)) = 2\sqrt{-y}$ con condizione iniziale $y(0) = -1$, costruiamo l'approssimazione supponendo di essere arrivati ad integrare fino ad un punto x_0 e vogliamo proseguire di h:

1. come prima cosa si calcola la derivata nel punto x_0, $y'(x_0) = f(x,\, y(x))_{|x=x_0}$, come si vede in verde in figura;
2. ci si sposta di $k_1 = hf(t,\, y)$ arrivando nel punto di ascissa $x_f = x_0 + h$ ma non lungo la soluzione poichè si è proseguiti linearmente lungo la retta verde, qui si calcola $k_2 = hf(t_n + h,\, y_n + k_1)$, come si vede in magenta in figura;
3. si prende la media dei due spostamenti sulle ordinate k_1 e k_2:

$$\boxed{y_{n+1} = y_n + \frac{k_1 + k_2}{2}} \quad .$$

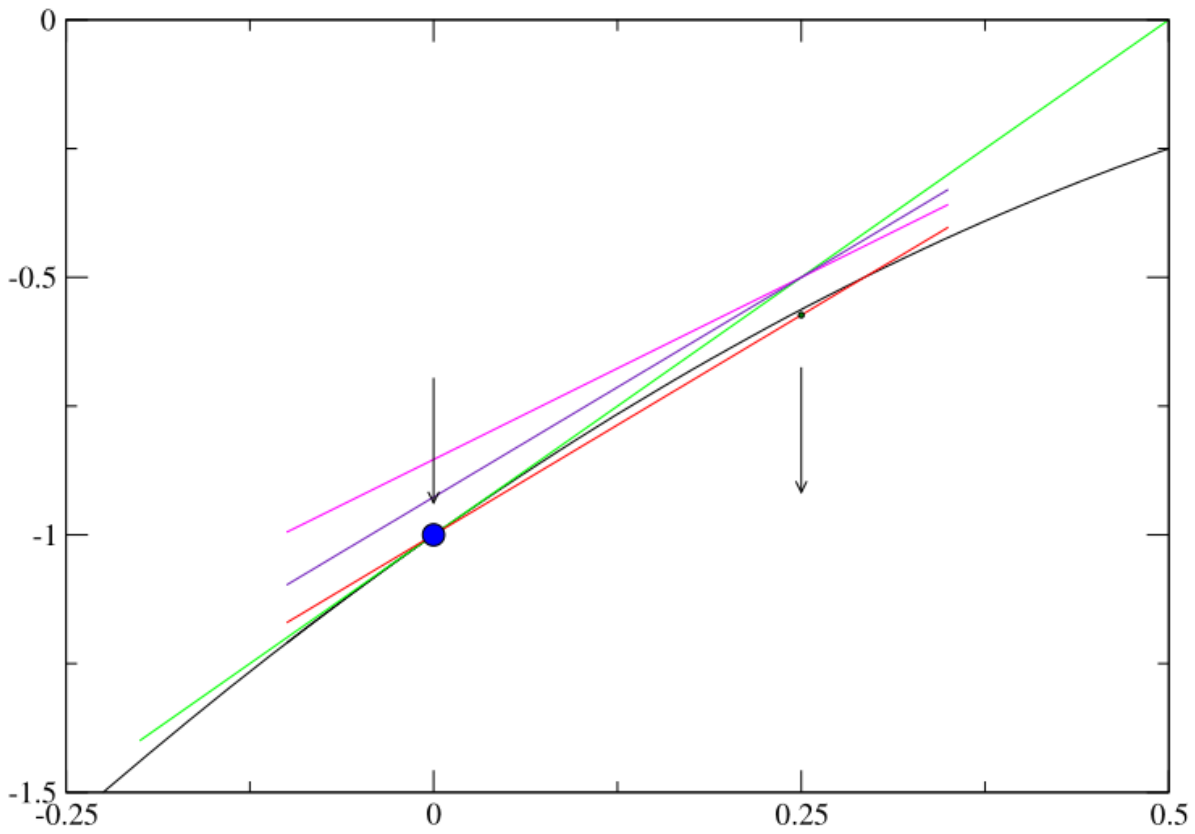


FIG. 4.6: esempio visuale di applicazione di Runge-Kutta

I passaggi spiegati sono la base del Runge-Kutta e corrispondono ad un metodo del secondo ordine.

Per ottenere ordini superiori, si introducono valutazioni intermedie della derivata. L'idea è approssimare sempre meglio il valore della pendenza media integrale, scegliendo opportunamente ascisse intermedie e combinazioni lineari delle derivate.

Un metodo RK di ordine quattro può utilizzare quattro valutazioni:

$$k_1 = hf(t_n, y_n),$$

$$k_2 = hf\big(t_n + \tfrac{h}{2},\, y_n + \tfrac{k_1}{2}\big),$$

$$k_3 = hf\big(t_n + \tfrac{h}{2},\, y_n + \tfrac{k_2}{2}\big),$$

$$k_4 = hf(t_n + h,\, y_n + k_3).$$

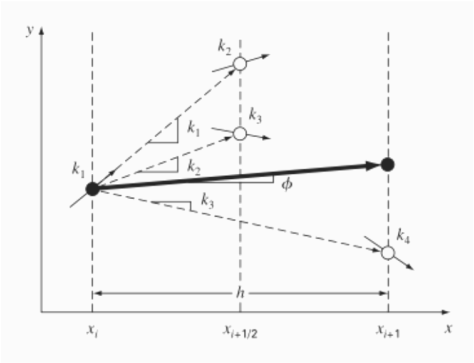


FIG. 4.7: Visualizzazione delle 4 valutazioni da fare in un RK4

Ogni k_i è una stima della derivata in un punto diverso dell'intervallo, costruita per compensare l'errore di espansione di Taylor del passo precedente. l valore numerico della soluzione viene aggiornato combinando i contributi:

$$\boxed{y_{n+1} = y_n + \frac{1}{6}\big(k_1 + 2k_2 + 2k_3 + k_4\big)} \quad . \tag{4.6}$$

Questa particolare combinazione lineare è scelta in modo che l'errore globale sia dell'ordine h^4, e che l'espansione in serie di Taylor del metodo coincida con quella della soluzione esatta fino ai termini di ordine h^4. Il metodo risultante è il molto utilizzato *Runge–Kutta del quarto ordine* (**RK4**).

> Ricorda le parole del Mannella: Runge-Kutta va usato con passo temporale variabile, a passo fisso fa schifo!

Controllo del passo mediante Richardson

Una tecnica classica per stimare l'errore locale di un metodo one-step consiste nell'applicare lo stesso algoritmo con due passi diversi, h e $2h$, e confrontare i risultati. Sia $Y(x_n)$ la soluzione esatta e $y_h(x_n)$ l'approssimazione ottenuta con passo h e $y_{2h}(x_n)$ quella ottenuta con passo $2h$. Per un metodo di ordine due (come il trapezoidale), vale lo sviluppo:

$$Y(x_n) - y_h(x_n) = D(x_n)h^2 + O(h^3), \qquad Y(x_n) - y_{2h}(x_n) = 4D(x_n)h^2 + O(h^3).$$

Eliminando il coefficiente ignoto $D(x_n)$ si ottiene un'approssimazione più precisa:

$$Y(x_n) \;\approx\; \frac{4y_h(x_n) - y_{2h}(x_n)}{3} + O(h^3) \quad ,$$

quindi una stima dell'errore è data da:

$$Y(x_n) - y_h(x_n) \;\approx\; \frac{y_h(x_n) - y_{2h}(x_n)}{3} \quad .$$

Questa informazione permette di regolare dinamicamente il passo di integrazione per mantenere l'errore entro una tolleranza prefissata. L'unico limite pratico è che, per stimare l'errore, occorre disporre sia dell'integrazione a passo h sia di quella a passo $2h$.

x	$y_h(x)$	$y_{2h}(x)$	Errore stimato	$Y(x) - y_h(x)$
1.0	0.49602	0.48314	0.00429	0.00398
2.0	0.33099	0.32361	0.00246	0.00234
3.0	0.24852	0.24389	0.00154	0.00148
4.0	0.19899	0.19484	0.00105	0.00101
5.0	0.16594	0.16366	0.00076	0.00073

TAB. 4.3: Stima dell'errore tramite Richardson (Atkinson, $y' = -y^2$, $y(0) = 1$, $h = 0.25$).

Il metodo del midpoint modificato

Il metodo del midpoint modificato è un algoritmo one-step composto che permette di muovere la soluzione da x a $x + H$ suddividendo l'intervallo in n sotto-passi di ampiezza $h = H/n$. Si introducono le variabili ausiliarie z_m, con:

$$z_0 = y(x), \qquad z_1 = z_0 + hf(x, z_0),$$

e la ricorrenza centrale:

$$z_{m+1} = z_{m-1} + 2h\, f(x + mh, z_m), \qquad m = 1, 2, \ldots, n-1.$$

Il valore finale è ottenuto tramite:

$$y(x + H) \approx y_n = \frac{1}{2}\big(z_n + z_{n-1} + hf(x + H,\, z_n)\big).$$

L'aspetto notevole di questo metodo è che l'errore dipende soltanto da potenze pari di h, cioè:

$$y(x + H) - y_n = \sum_{k=1}^{\infty} \alpha_k h^{2k}.$$

Questa proprietà lo rende particolarmente adatto all'accoppiamento con metodi di estrapolazione, riducendo drasticamente il numero di valutazioni della funzione richiesta per ottenere alta accuratezza.

Richardson extrapolation e metodo di Burlisch–Stoer

La *Richardson extrapolation* si basa sull'idea che l'approssimazione ottenuta da un metodo one-step su un intervallo di ampiezza H dipende analiticamente dal passo elementare $h = H/n$. Applicando l'algoritmo elementare con $n = 1, 2, 4, 8, \ldots$ sotto-passi, si ottengono approssimazioni $y_1, y_2, y_4, y_8, \ldots$, che possono essere estrapolate nel limite $n \to \infty$ (ossia $h \to 0$) utilizzando una funzione razionale

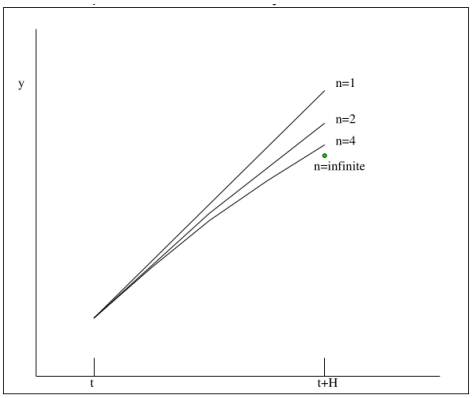


FIG. 4.8: Idea dietro l'estrapolazione di Richardson

$$\boxed{R_{i(i+1)\ldots(i+m)} = \frac{p_0 + p_1 x + \cdots + p_\mu x^\mu}{1 + q_1 x + \cdots + q_\nu x^\nu}}, \qquad x = \frac{H}{n}.$$

Questo è il risultato del **metodo di Burlisch-Stoer** che combina queste due idee:

1. usa il *midpoint modificato* come metodo elementare di integrazione su $[x, x + H]$;
2. applica la Richardson extrapolation ai risultati ottenuti per $n = 1, 2, 4, \ldots$;
3. costruisce una matrice di estrapolazione (tabella di Richardson) da cui selezionare la stima più accurata.

La scelta del passo H e del numero di sotto-passaggi n è adattata dinamicamente per minimizzare il costo computazionale mantenendo al contempo l'errore entro limiti prestabiliti. Il metodo Burlisch–Stoer è estremamente efficiente per funzioni sufficientemente regolari ed è in grado di raggiungere ordini molto elevati, pur senza ricorrere a valutazioni multiple di derivate alte.

Metodi Runge-Kutta Hamiltoniani

Consideriamo un sistema Hamiltoniano del tipo: $\boxed{H(q,p) = \frac{p^2}{2} + V(q)}$, con equazioni del moto:

$$\dot{q} = \frac{\partial H}{\partial p}\,, \qquad \dot{p} = -\frac{\partial H}{\partial q}\,,$$

e date certe condizioni al contorno $p(0)$ and $q(0)$. Vista la forma di RK4 pensiamo di potervi costruire una *contact transformation*, ovvero una mappa da q, p a Q, P dove questi ultimi possono essere pensati come un evoluzione a un tempo successivo di q, p.

L'evoluzione generata da tali equazioni è una trasformazione *simplettica*, cioè preserva la struttura geometrica del flusso Hamiltoniano, come le trasformazioni canoniche studiate a meccanica classica (vedi capitolo settimo di '*Fisica teorica I: meccanica*' di Landau-Lifšits).

I metodi numerici come i Runge-Kutta classici, non sono in generale simplettici e possono introdurre deriva energetica nelle integrazioni di lungo periodo.

Idea generale. Un metodo Runge–Kutta Hamiltoniano (o simplettico) cerca di costruire una mappa

$$(q_n, p_n) \longmapsto (q_{n+1}, p_{n+1})$$

che sia a sua volta una trasformazione simplettica e che approssimi il flusso esatto dopo un passo h.

Per un metodo RK con coefficienti $\{a_i, b_i\}$, la trasformazione RK simplettica di ordine n assume la forma:

$$p_j = p_{j-1} + hb_j\, F(q_{j-1}), \qquad q_j = q_{j-1} + ha_j\, p_j,$$

che non sono nient'altro che l'implementazione delle equazioni di Hamilton, dove $F(q) = -V'(q)$ e ricordiamo di avere delle condizioni iniziali p_0, q_0 con cui andiamo a valutare p_n, q_n come punti evoluti dopo un tempo h, l'aggiornamento 'sfalsato' per cui p_j dipende dai p_{j-1}, q_{j-1} precedenti ma q_j dipende da q_{j-1}, p_j serve a mantenere la simpletticità.

'Velocity Verlet' / 'Leapfrog' L'esempio più semplice è quello in cui prendiamo la forma appena scritta per q_1, p_1, q_2, $p_a ncora2$ ed inseriamo i coefficienti $a_1 = a_2 = \frac{1}{2}$, $b_1 = 0$, $b_2 = 1$:

$$\begin{aligned}
p_1 &= p_0 + h \times 0 \times F(q_0) &= p_0 \\
q_1 &= q_0 + h \times \frac{1}{2} \times p_1 &= q_0 + \frac{h}{2} p_1 \\
p_2 &= p_1 + h \times 1 \times F(q_1) &= p_0 + hF(q_1) \\
q_2 &= q_1 + h \times \frac{1}{2} \times p_2 &= q_1 + \frac{h}{2} p_2
\end{aligned}$$

Interpretazione: q avanza in due metà simmetriche ($a_1 = a_2 = \frac{1}{2}$), mentre la p viene aggiornata solo a seguito di un mezzo passo in avanti.

Esempio di schema simplettico. Una scelta possibile di coefficienti *magici* è:

i	(a_i, b_i)
1	$(0.515352837431123,\ 0.134496199277431)$
2	$(-0.085782019412974,\ -0.224819803079421)$
3	$(0.441583023616467,\ 0.756320000515668)$
4	$(0.128846158365384,\ 0.334003603286321)$

Esempio pratico. Per il sistema $H(q,p) = \frac{p^2}{2} + \frac{q^4}{4} - \frac{q^2}{2}$, la comparazione tra un RK standard e un RK Hamiltoniano mostra che il metodo simplettico conserva l'Hamiltoniana in modo molto più accurato nel tempo, evitando il drift energetico tipico dei metodi non-simplettici.

4.0.3 Equazioni stiff

Un'equazione differenziale si dice *stiff* quando presenta scale temporali molto diverse: la soluzione varia lentamente su larga scala, ma la dinamica interna dell'equazione contiene componenti che decadono estremamente velocemente. In questi casi, i metodi espliciti richiedono passi h molto piccoli per ragioni di stabilità, anche quando la soluzione vera non lo richiederebbe.

Esempio semplice. Immaginiamo di dover risolvere un problema come un sistema

$$\begin{cases} u'(x) = 998\,u(x) + 1998\,v(x)\,, & u(0) = 1 \\ v'(x) = -999\,u(x) - 1999\,v(x)\,, & v(0) = 0 \end{cases}$$

Si può notare che le soluzioni analitiche esistono e sono $u(x) = 2e^{-x} - e^{-1000x}$ e $v(x) = -e^{-x} + e^{-1000x}$, ma dovendo risolvere tramite metodi numerici finiremmo per usare time step troppo piccoli per sopperire alla presenza di modi con tempi scala molto lontani fra loro, lo si vede subito scrivendo la matrice del sistema

$$A = \begin{pmatrix} 998 & 1998 \\ -999 & -1999 \end{pmatrix} \qquad \lambda_{1,2} = -1,\ -1000$$

Come affrontare il problema? Affrontiamo il problema grazie a metodi impliciti/semi-impliciti ad ordine basso, in particolare presa $y' = f(x,\, y(x))$, si linearizza un metodo di Euler implicito:

$$y_{n+1} = y_n + h\, f(x_{n+1},\, y_{n+1})$$

$$y_{n+1} = y_n + h\left[f(x_{n+1},\, y_n) + \frac{\partial f}{\partial y_n}(y_{n+1} - y_n)\right]$$

Portando a sinistra il termine con y_{n+1} e dividendo i due membri rimane:

$$y_{n+1} = \left[1 - h\,\frac{\partial f}{\partial y_n}\right]^{-1}\left(y_n + h\, f(x_{n+1}, y_n) - h\,\frac{\partial f}{\partial y_n} y_n\right)$$

Formalmente, definita $\mathbf{A}$ la matrice del nostro sistema lineare (matrice definita positiva, $\lambda_i > 0 \; \forall \, i$) e dato $\mathbf{y}' = -\mathbf{A}\,\mathbf{y}$, utilizzando un metodo di eulero esplicito abbiamo:

$$\mathbf{y}_{n+1} \;=\; \mathbf{y}_n + h\mathbf{y'}_n \;=\; (1 - h\,\mathbf{A})\,\mathbf{y}_n \;\equiv\; \mathbf{B}\,\mathbf{y}_n$$

e $\mathbf{B}^n \to 0$ per $n \to \infty$ solo se gli autovalori $(1 - \lambda_i h) \equiv \mu_i$ di $\mathbf{B}$ sono < 1 e questo tipicamente non accade per le *stiff equations*.

Tuttavia, usando un metodo implicito che assume la forma $\boxed{(Id + h\mathbf{A})\mathbf{y}_{n+1} = \mathbf{y}_n}$ da non confondere con il metodo esplicito appena sviluppato sopra, abbiamo una diversa $\mathbf{B}$: $\mathbf{y}_{n+1} = (Id + h\mathbf{A})^{-1}\mathbf{y}_n \equiv \mathbf{B}\mathbf{y}_n$. Se gli autovalori di $\mathbf{A}$ sono $\lambda > 0$, allora gli autovalori di $(Id + h\mathbf{A})^{-1}$ sono $\mu_i \equiv (1 + h\lambda)^{-1} \; \forall \, h$ quindi la soluzione si comporta bene $(0 < \mu_i < 1)$.

4.0.4 Boundary Value Problems (BVP)

Prendiamo di esempio un'equazione di un oscillatore armonico smorzato $y'' = -\gamma y' - \omega_0^2 y$ con una condizione iniziale $y(0) = 1$ ed un'altra condizione al tempo finale $t_f = k\pi/\omega$, dove $\omega^2 \equiv \omega_0^2 - \gamma^2/4$.

Il problema che si verifica è che data la tipica soluzione $y(t) = e^{-\gamma t/2}(A\,sin(\omega t) + B\,cos(\omega t))$, mentre la condizione iniziale ci porta ad imporre $A = 0$, la condizione sul punto finale non porta alcuna informazione in più per B perché abbiamo scelto il punto finale in un nodo dell'oscillazione portandoci ad avere $y(t_f) = e^{-\gamma t/2}(-1)^k$ che non contiene né B né A quindi non ci aiuta a trovare una soluzione univoca.

Abbiamo quindi un grosso problema: *la soluzione non è unica!*

Vediamo due diversi modi per affrontare il problema.

Shooting method

Come si può immaginare dal nome, qui *spariamo* a caso.

Ripartiamo dallo stesso problema di oscillatore armonico smorzato e immaginiamo di voler imporre la condizione al tempo finale $y(t_f) = y(5.02) = 0.418$.

Diamo dei valori alle costanti, come $\gamma = 0.3$ ed $\omega_0 = 1$, a questo punto siamo costretti per costruire una simil-soluzione a imporre un'altra condizione iniziale, come la derivata $y'(0) = s$, fatto ciò integriamo fino al punto finale.

Ripetiamo la costruzione per diversi valori di s fino ad averne uno che approssimi bene la condizione che avremmo voluto imporre: $y(5.02,\, s) \approx 0.418$.

metodo di rilassamento / differenze finite

Il metodo consiste nel mettere un tentativo di soluzione, *initial guess*, fra x_0 ed x_f e 'rilassarlo' fino a trovare la soluzione corretta.

Partiamo ad esempio da una $y'' = f(x, y, y')$ con condizioni al bordo $y(x_0) = g_0$ e $y(x_n) = g_n$. Si assume una mesh di n punti spaziati di un passo h, usando le formule delle differenze finite possiamo scrivere:

$$\frac{y_{n-1} - 2y_n + y_{n+1}}{h^2} \;=\; f(x_n,\, y_n,\, \frac{y_{n+1} - y_{n-1}}{2h}) \;\equiv\; f_n$$

dove y_0 e y_n sono condizioni date in precedenza, riscriviamo in forma matriciale il problema:

$$\frac{1}{h^2}\begin{pmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \cdots & 0 \\ 0 & 1 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & -2 & 1 \\ 0 & \cdots & 0 & 1 & -2 \end{pmatrix}\begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ \\ f_{N-1} \end{pmatrix} - \begin{pmatrix} g_0/h^2 \\ 0 \\ \vdots \\ 0 \\ g_n/h^2 \end{pmatrix}$$

che può essere scritto come $\frac{1}{h^2}\mathbf{A}Y \equiv \mathbf{F}(Y) + \mathbf{G}$ e da questa forma si risolve con metodo di Newton dopo aver linearizzato F(Y):

$$\mathbf{y}^{(m+1)} \;=\; \mathbf{y}^{(m)} \;-\; \left[\frac{\mathbf{A}}{h^2} - \frac{\partial \mathbf{F}_i}{\partial y_j}\right]^{-1} \left[\frac{\mathbf{A}y^m}{h^2} - \mathbf{F}(y^{(m)}) - \mathbf{G}\right]$$

A questo punto possiamo dare un guess iniziale $y^{(0)}$ ed iterare.

Capitolo 5

Integrazione numerica di equazioni differenziali alle derivate parziali (PDEs)

5.1 Introduzione

Esistono principalmente tre tipi di equazioni differenziali alle derivate parziali (PDE, *partial differential equation*), la differenza computazionalmente sta nel caso in cui abbiamo **valori iniziali** (problema principale: stabilità) o utilizziamo **condizioni al contorno** (problema principale: efficienza):

- **Iperboliche**: come ad esempio l'equazione delle onde

$$\boxed{\frac{\partial^2 u}{\partial t^2} - v^2 \frac{\partial^2 u}{\partial x^2} \;=\; 0} \quad ; \tag{5.1}$$

- **Paraboliche**: come ad esempio l'equazione di diffusione

$$\boxed{\frac{\partial P}{\partial t} - \frac{\partial}{\partial x}\left(D \frac{\partial}{\partial x} P\right) \;=\; 0} \quad ; \tag{5.2}$$

- **Ellittiche**: come ad esempio l'equazione di Poisson

$$\boxed{\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \;=\; \rho(x,y)} \quad . \tag{5.3}$$

applicazione

Mettiamo che io prenda $\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \rho(x,y)$, prima cosa che vorremmo fare è introdurre uno *spacing* Δ in modo che $x_j = x_0 + j\Delta$ (con j=0, ..., J) e $y_\ell = y_0 + j\Delta$ (con ℓ=0, ..., L).

Possiamo esprimere le derivate come spiegato nel paragrafo sulle differenze finite:

$$\frac{u_{j+1,\ \ell} - 2u_{j,\ \ell} + u_{j-1,\ \ell}}{\Delta^2} + \frac{u_{j,\ \ell+1} - 2u_{j,\ \ell} + u_{j,\ \ell-1}}{\Delta^2} \ = \ \rho(x,y)$$

Il modo di soluzione che introduciamo ora passa attraverso l'introdurre un indice $i = j(L+1) + \ell$ (per j=0 va da 0 ad L, per j=1 va da L+1 a 2L+1 e così via...) così che u diventi un vettore ed il problema diventi un problema di algebra lineare:

$$u_{i+L+1} + u_{i-(L+1)} + u_{i+1} + u_{i-1} - 4u_i \ = \ \Delta^2 \rho(x,y) \quad ,$$

$$\mathbf{A}\ \vec{u} \ = \ \vec{b}$$

dove in questo caso ci si può convincere che

$$\mathbf{A} = \left[\begin{array}{ccc|ccc|ccc}
-4 & 1 & & 1 & & & & & \\
1 & -4 & 1 & & 1 & & & & \\
 & 1 & -4 & & & 1 & & & \\
\hline
1 & & & -4 & 1 & & 1 & & \\
 & 1 & & 1 & -4 & 1 & & 1 & \\
 & & 1 & & 1 & -4 & & & 1 \\
\hline
 & & & 1 & & & -4 & 1 & \\
 & & & & 1 & & 1 & -4 & 1 \\
 & & & & & 1 & & 1 & -4
\end{array}\right]$$

Come risolvere $\mathbf{A}\vec{u} = \vec{b}$?

- quando $\mathbf{A}$ è costante con Fast Fourier methods;
- direttamente, usando ad esempio il gradiente coniugato (molto accurato, non molto efficiente quando $\mathbf{A}$ è grande perché 'sparsa');
- Rilassamento, scrivendo $\mathbf{A} = \mathbf{E} - \mathbf{F}$ dove $\mathbf{E}$ è invertibile, si inizia da un guess $\vec{u}^0$ e si itera risolvendo $\mathbf{E}\vec{u}^{(r+1)} = \vec{b} + \mathbf{F}\vec{u}^{(r)}$

5.1.1 Differenze finite

Il metodo delle differenze finite è una strategia utilizzata per risolvere numericamente equazioni differenziali che, nelle sue varianti, si basa sull'approssimazione delle derivate con equazioni alle differenze finite. Viene utilizzato prevalentemente per equazioni differenziali ordinarie, anche se il metodo viene sfruttato come schema di avanzamento nel tempo per problemi alle derivate parziali.

$$u_{i+1} = u_i + u_i' dx + \frac{1}{2} u_i'' dx^2 + ...$$

$$\frac{u_{i+1} - u_i}{dx} = u_i' + \frac{1}{2} u_i'' dx + ...$$

$$u_{i+1} - u_{i-1} = 2u_i' dx + \frac{1}{6} u_i''' dx^3$$

Che cosa vuol dire? che la derivata nel punto i-esimo può essere in qualche modo ricavata attraverso valori della funzione u in i e nei punti vicini i+1, i-1, etc. .
Possiamo poi stimare l'errore di queste stime grazie alle serie di Taylor.

$$u'_{i=0} = \frac{1}{2dx}\left(-u_{-1} + u_1\right) + ...dx^2 u'''_0 \qquad u'_0 = \frac{1}{12dx}\left[u_{-2} - 8u_{-1} - 8u_1 - u_2\right] + ...dx^4 u_0^{(5)}$$

$$\text{con al bordo:} \quad u'_0 = \frac{1}{dx}\left(-u_0 + u_1\right) \quad \text{oppure} \quad u'_0 = \frac{1}{2dx}(-3u_0 + 4u_1 - u_2)$$

$$u''_0 = \frac{1}{dx^2}\left(u_{-1} - 2u_0 + u_1\right) + ...dx^2 u_0^{(4)} \qquad u''_0 = \frac{1}{12dx^2}\left(-u_{-2} + 16u_{-1} - 30u_0 + 16u_1 - u_2\right) + ...dx^2 y_0^{(6)}$$

$$\text{con al bordo:} \quad u''_0 = \frac{1}{dx^2}(-2u_0 - 5u_1 + 4u_2 - u_3)$$

5.1.2 Flux-conservative PDEs

Molte equazioni differenziali sono nella forma di un'equazione che conserva il flusso:

$$\boxed{\frac{\partial}{\partial t}\mathbf{u} \;=\; -\frac{\partial}{\partial x}\mathbf{F}(\mathbf{u})} \quad , \tag{5.4}$$

dove $\mathbf{F}(u)$ è il flusso della quantità.
Ad esempio, anche l'equazione standard iperbolica $\frac{\partial^2 u}{\partial t^2} = v^2 \frac{\partial^2 u}{\partial x^2}$ può essere scritta, definendo $s \equiv \frac{\partial}{\partial t}u$ e $r \equiv v\frac{\partial}{\partial x}u$, come un'equazione del tipo sopra definito:

$$\frac{\partial}{\partial t}s = v\frac{\partial}{\partial x}r \quad , \qquad \frac{\partial}{\partial t}r = v\frac{\partial}{\partial x}s \quad ,$$

definendo $s \equiv \frac{\partial}{\partial t}u$ e $r \equiv v\frac{\partial}{\partial x}u$.

Forward Time, Centered Space

Altro esempio di PDS che possiamo scrivere in forma flux-conservative è $\boxed{\frac{\partial u}{\partial t} = v\frac{\partial u}{\partial x}}$ se considerato $\mathbf{F}(u) = vu$. Prendiamo appunto $\frac{\partial u}{\partial t} = v\frac{\partial u}{\partial x}$, usando $x_j = x_0 + j\Delta x$ (j=0, ..., J) e $t_n = t_0 + n\Delta t$ (n=0, ..., N) andiamo a discretizzare $u_j^n \equiv u(t_n, x_j)$ e ancora una volta esprimiamo le derivate a meno di un errore:

$$\frac{\partial u}{\partial t}|_{j,n} = \frac{u_j^{n+1} - u_j^n}{\Delta t} + O(\Delta t) \qquad \frac{\partial u}{\partial x}|_{j,n} = \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} + O(\Delta x^2)$$

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = -v \left(\frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} \right) \tag{5.5}$$

A partire da eq.4.11 sviluppiamo la soluzione chiamata *forward time centered space* (**FTCS**), forma esplicita in cui esprimiamo i valori 'forward' u^{n+1} nel tempo in funzione di u^n e centrata nello spazio:

$$u_j^{n+1} = u_j^n - v\Delta t \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} \quad . \tag{5.6}$$

Possiamo fare un'**analisi di stabilità di Von Neumann** arrivati a questo punto: l'idea è quella di studiare la stabilità lineare di un'onda, assumendo $u_j^n = e^{i(kx-\omega t)}$ e definito $\xi \equiv e^{i\omega\Delta t}$ si ha $u_j^n = \xi^n e^{ikj\Delta x}$ dove ξ^n rappresenta l'ampiezza al tempo t_n.
Sostituendo u_j^n in eq.4.12 si ottiene:

$$\xi = 1 + i\frac{v\Delta t}{\Delta x} sin(k\Delta x) \quad .$$

Risulta chiaro che $|\xi| > 1$ quindi secondo quest'analisi per ogni scelta di Δx e Δt il metodo è instabile $\forall$ k.

Metodo di Lax

Il metodo di Lax consiste nel sostituire $u_j^n \to \frac{u_{j+1}^n - u_{j-1}^n}{2}$ all'interno di eq.4.12 così da fare un'analisi di stabilità di Von Neumann sulla forma:

$$u_j^{n+1} = \frac{u_{j+1}^n - u_{j-1}^n}{2} + v\Delta t \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} \quad .$$

$$\xi = cos(k\Delta x) - i\frac{v\Delta t}{\Delta x} sin(k\Delta x)$$

dove abbiamo usato ancora $u_j^n = \xi^n e^{ikj\Delta x}$.

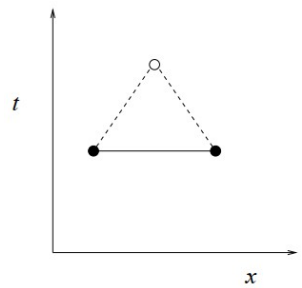


FIG. 5.1: Caption

Ora per la stabilità si ottiene $|\xi|^2 = cos^2(k\Delta x) + \left(\frac{v\Delta t}{\Delta x}\right)^2 sin^2(k\Delta x)$, che può essere stabile ($|\xi| < 1$) se viene rispettato il vincolo $\frac{|v|\Delta t}{\Delta x} \leq 1$

5.1.3 Una soluzione pratica

Partiamo dall'idea di risolvere un'equazione differenziale come ad esempio l'importante **equazione di Burgers**

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0 \quad ,$$

alla quale in più potremmo aggiungere un termine $\nu\frac{\partial^2 u}{\partial x^2}$ che rappresenti la viscosità.
Immaginiamo di poter linearizzare $u = u_0 + \delta u$ tale che $\langle u \rangle = u_0$ e $|\delta u| \ll |u_0|$.

$$u\frac{\partial u}{\partial x} = (u_0 + \delta u)\frac{\partial}{\partial x}(u_0 + \delta u) = u_0 u_0' + u_0 \delta u' + \delta u u_0' + \delta u \delta u'$$

Riconduciamo l'equazione a:

$$\frac{\partial(\delta u)}{\partial t} + u_0 \frac{\partial(\delta u)}{\partial x} + \cancel{\delta u \frac{\partial u_0}{\partial x}} = 0$$

Cosa significherebbe risolvere questa equazione differenziale? innanzitutto nella pratica divideremo lo spazio discretamente e di conseguenza la u allo stesso modo.
Andiamo a supporre una soluzione $u(x,t) = U(x - ct)$ che si propaga a velocità costante.

Se noi andiamo a scrivere un programma per far propagare un'onda potremmo usare ciò che abbiamo visto nella sezione su *Forward Time Centered Space*.

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = -u\left(\frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x}\right)$$

Vediamo un esempio di programma in c++ con u=1 di fronte la derivata spaziale:

```
#include <iostream>
#include <fstream>
#include <cmath>

using namespace std;

// EQUAZIONE DEL TIPO:   du/dt = -c * du/dx

int main()  {

    const int N = 200;
    const int M = 2000;
    const double pi = 3.14159265358979323846;
    //float XX[N], tt[M];
    double ff[M][N];
    const double L = 2*pi, dx= L / N;
    const double T = 10.0, dt= T / M;

    for(int j=0; j<N; j++){                    // carico inizialmente la condizione iniziale u(x, t=0)
        ff[0][j] = sin( 2* j * dx ) +  sin( 10 * j * dx ) / 8 ;
    }

    /* questa è la nostra funzione al tempo t=0 per ogni x
     andiamo a calcolare l'evoluzione temporale con
     forward time e centered space*/

    for (int k = 1; k < M-1; k++){

```

```
29
30          for (int i = 1; i < N-1; i++){
31              // Ho spostato la derivata direttamente qui dentro
32              double ksucc = (ff[k-1][i+1] - ff[k-1][i-1]) / (2
                    * dx);
33
34              ff[k][i] = ff[k-1][i] + dt*ksucc;
35          }
36
37          // condizioni al contorno
38          ff[k][0] = ff[k-1][0] + dt * ( ff[k-1][1] - ff[k-1][N
                -1] ) / ( 2 * dx );
39          ff[k][N-1] = ff[k-1][N-1] + dt * ( ff[k-1][0] - ff[k
                -1][N-2] ) / ( 2 * dx );
40      }
41
42      // SALVATAGGIO SU FILE PER PYTHON
43      ofstream file("output.txt");
44      if (!file) {
45          cout << "Errore apertura file!\n";
46          return 1;
47      }
48
49      for (int k = 0; k <= M; ++k) {
50          for (int i = 0; i < N; ++i) {
51              file << ff[k][i] << "\n";
52          }
53          file << "\n";
54      }
55
56      file.close();
57      cout << "File output.txt generato!\n";
58
59      return 0;
60  }
```

Listing 5.1: Burgers: applicazione all'evoluzione di un'ode con FTCS

Aggiungiamo il termine di dissipazione per ottenere $\boxed{\frac{\partial u}{\partial t} = -u\frac{\partial u}{\partial x} + \nu\frac{\partial^2 u}{\partial x^2}}$ così che la eq.4.12 diventi questa volta:

$$u_j^{n+1} = u_j^n + \Delta t\frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} + \nu\Delta t\frac{u_{j-1}^n - 2u_j^n + u_{j+1}^n}{dx^2} \quad .$$

```
1  #include <iostream>
2  #include <fstream>
3  #include <cmath>
4
5  // EQUAZIONE DEL TIPO:   du/dt + c * du/dx = d^2u/dx^2
```

```
6
7  using namespace std;
8  const int N = 50;
9
10 // Derivata prima / differenze finite / condizioni al contorno
       periodiche
11 inline double derivata_prima(double ff[][N], int k, int i,
       double dx, int N){
12     int ip = (i + 1) % N;
13     int im = (i - 1 + N) % N;
14
15     return (ff[k][ip] - ff[k][im]) / (2.0 * dx);
16 }
17
18 // Derivata seconda / differenze finite / condizioni al
       contorno periodiche
19 inline double derivata_seconda(double ff[][N], int k, int i,
       double dx, int N){
20     int ip = (i + 1) % N;
21     int im = (i - 1 + N) % N;
22
23     return (ff[k][ip] - 2.0 * ff[k][i] + ff[k][im]) / (dx * dx
           );
24 }
25
26 // Avanzamento in avanti nel tempo
27 inline double forward_time(double u, double dudx, double dt,
       double c ){
28     return u + dt * dudx;
29 }// il segno meno per il termine du/dx è stato incluso nella
       chiamata della funzione
30
31 int main()  {
32     const int M = 1000;
33     const double pi = 3.14159265358979323846;
34
35     static double ff[M][N];
36     const double L = 2*pi, dx= L / N;
37     const double T = 10, dt= T / M;
38     double v = 0.008; /*viscosità cinematica*/
39
40     for(int j=0; j<N; j++){
41         ff[0][j] = sin( 2* j * dx );
42     }
43
44     for (int k = 1; k < M; k++){
45
46         for (int i = 0; i < N; i++){
47             double ksucc = (-derivata_prima(ff, k-1, i, dx, N)
                   + v*derivata_seconda(ff, k-1, i, dx, N));
48
```

```
            ff[k][i] = forward_time(ff[k-1][i], ksucc, dt,
                1.0);
        }

    }

    // SALVATAGGIO SU FILE PER PYTHON
    ofstream file("output.txt");
    if (!file) {
        cout << "Errore apertura file!\n";
        return 1;
    }

    for (int k = 0; k < M; ++k) {
        for (int i = 0; i < N; ++i) {
            file << ff[k][i] << "\n";
        }
        file << "\n";
    }

    file.close();
    cout << "File output.txt generato!\n";

    return 0;
}
```

Listing 5.2: upgrade dell'applicazione precedente

Appendice A

Metodo delle differenze finite

Introduzione

Il metodo delle differenze finite è una tecnica numerica utilizzata per approssimare le derivate di una funzione mediante valori della stessa in punti discreti.

Conosciamo bene la definizione di derivata come rapporto incrementale, possiamo quindi usare punti discreti della funzione per svolgere questo rapporto ed approssimare la funzione:

$$\frac{d\varphi}{dx} = \lim_{h\to 0} \frac{\varphi(x+h) - \varphi(x)}{h} \qquad \text{rapporto incrementale in avanti}$$

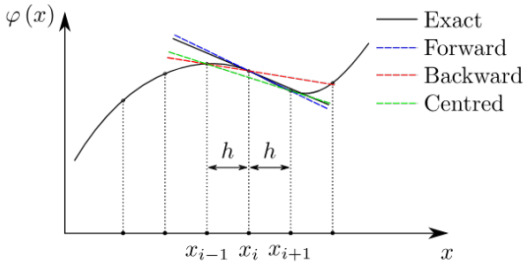
$$\frac{d\varphi}{dx} = \lim_{h\to 0} \frac{\varphi(x) - \varphi(x-h)}{h} \qquad \text{rapporto incrementale allíndietro}$$

$$\frac{d\varphi}{dx} = \lim_{h\to 0} \frac{\varphi(x+h) - \varphi(x-h)}{2\,h} \qquad \text{rapporto incrementale centrato}$$

Questo metodo ci dice che considerando il dominio $x \in [0, X]$ e ponendo la *mesh* di N punti, tale che l'intervallo sia $h = X/N$, l'approssimazione della derivata calcolata con punti discreti della funzione può essere calcolata con:

$$\frac{d\varphi}{dx}|_i \approx \frac{\varphi_{i+1} - \varphi_i}{h}\ , \quad \frac{d\varphi}{dx}|_i \approx \frac{\varphi_i - \varphi_{i-1}}{h}\ , \quad \frac{d\varphi}{dx}|_i \approx \frac{\varphi_{i+1} - \varphi_{i-1}}{2\,h}\ . \tag{A.1}$$

Nessuna di queste formule è migliore delle altre in ogni occasione, per ogni funzione che si voglia derivare.



Nel caso in figura si vede come il rapporto in avanti è qui il migliore per approssimare la derivata prima.

Fig. A.1: Prova di applicazione del metodo delle differenze finite al calcolo della derivata prima di una generica funzione polinomiale.

Accuratezza delle approssimazioni

Per studiare l'errore utilizziamo lo sviluppo di Taylor.

$$\varphi(x) = \varphi(x_0) + (x - x_0)\left.\frac{d\varphi}{dx}\right|_{x_0} + \frac{(x-x_0)^2}{2}\left.\frac{d^2\varphi}{dx^2}\right|_{x_0} + \frac{(x-x_0)^3}{6}\left.\frac{d^3\varphi}{dx^3}\right|_{x_0} + \dots$$

Ponendo $x_i = x_0$ otteniamo per φ_{i+1} , φ_{i-1}:

$$\varphi_{i+1} = \varphi_i + (x_{i+1} - x_i)\left.\frac{d\varphi}{dx}\right|_i + \frac{(x_{i+1}-x_i)^2}{2}\left.\frac{d^2\varphi}{dx^2}\right|_i + \frac{(x_{i+1}-x_i)^3}{6}\left.\frac{d^3\varphi}{dx^3}\right|_i + \dots$$

$$\varphi_{i-1} = \varphi_i + (x_{i-1} - x_i)\left.\frac{d\varphi}{dx}\right|_i + \frac{(x_{i-1}-x_i)^2}{2}\left.\frac{d^2\varphi}{dx^2}\right|_i + \frac{(x_{i-1}-x_i)^3}{6}\left.\frac{d^3\varphi}{dx^3}\right|_i + \dots$$

Poiché $x_{i+1} - x_i = h$, si ottiene

$$\begin{aligned}\varphi_{i+1} &= \varphi_i + h\left.\frac{d\varphi}{dx}\right|_i + \frac{h^2}{2}\left.\frac{d^2\varphi}{dx^2}\right|_i + \frac{h^3}{6}\left.\frac{d^3\varphi}{dx^3}\right|_i + \dots \\ \varphi_{i-1} &= \varphi_i - h\left.\frac{d\varphi}{dx}\right|_i + \frac{h^2}{2}\left.\frac{d^2\varphi}{dx^2}\right|_i - \frac{h^3}{6}\left.\frac{d^3\varphi}{dx^3}\right|_i + \dots\end{aligned} \tag{A.2}$$

Da queste espansioni in eq.(A.2) possiamo riottenere la derivata con metodo forward e backward:

$$\begin{aligned}\left.\frac{d\varphi}{dx}\right|_i &= \frac{\varphi_{i+1}-\varphi_i}{h} - \frac{h}{2}\left.\frac{d^2\varphi}{dx^2}\right|_i - \frac{h^2}{6}\left.\frac{d^3\varphi}{dx^3}\right|_i + \dots = \\ &= \frac{\varphi_{i+1}-\varphi_i}{h} + O(h) \qquad (\textbf{forward}) \\ \left.\frac{d\varphi}{dx}\right|_i &= \frac{\varphi_i-\varphi_{i-1}}{h} + \frac{h}{2}\left.\frac{d^2\varphi}{dx^2}\right|_i - \frac{h^2}{6}\left.\frac{d^3\varphi}{dx^3}\right|_i + \dots = \\ &= \frac{\varphi_i-\varphi_{i-1}}{h} + O(h) \qquad (\textbf{backward})\end{aligned} \tag{A.3}$$

Sommando le due espressioni in eq.(A.3) si ottiene

$$\left.\frac{d\varphi}{dx}\right|_i = \frac{\varphi_{i+1}-\varphi_{i-1}}{2h} - \frac{h^2}{6}\left.\frac{d^3\varphi}{dx^3}\right|_i + \dots = \frac{\varphi_{i+1}-\varphi_{i-1}}{2h} + O(h^2)$$

Questa formula ha un errore di ordine h^2, quindi è più accurata rispetto alle approssimazioni forward e backward che hanno errore $O(h)$.

Approssimazione della derivata seconda

Sottraendo le due espansioni in eq.(A.3) si ottiene

$$0 \;=\; \frac{\varphi_{i+1} - \varphi_i}{h} \;-\; \frac{\varphi_i - \varphi_{i-1}}{h} \;-\; h\,\frac{d^2\varphi}{dx^2}\bigg|_i \;+\; O(h^3)$$

da cui segue la formula standard della derivata seconda calcolata con il metodo delle differenze finite:

$$\frac{d^2\varphi}{dx^2}\bigg|_i \;=\; \frac{\varphi_{i+1} - 2\varphi_i + \varphi_{i-1}}{h^2} \;+\; O(h^2) \tag{A.4}$$

La stessa espressione può essere ottenuta partendo dalla definizione della derivata seconda, approssimando la derivata prima nei punti intermedi e sostituendo le differenze finite:

$$\begin{aligned}\frac{d^2\varphi}{dx^2}\bigg|_i \;&=\; \frac{d}{dx}\left(\frac{d\varphi}{dx}\right)_i \;=\; \lim_{h\to 0}\frac{\frac{d\varphi}{dx}\big|_{i+1/2} - \frac{d\varphi}{dx}\big|_{i-1/2}}{h} \;= \\ &\approx\; \frac{\frac{\varphi_{i+1}-\varphi_i}{h} - \frac{\varphi_i-\varphi_{i-1}}{h}}{h} \;=\; \frac{\varphi_{i+1} - 2\varphi_i + \varphi_{i-1}}{h^2}\end{aligned}$$

Appendice B

Algoritmo di Thomas

L'algoritmo di Thomas è un metodo numerico molto efficiente per la risoluzione di sistemi lineari la cui matrice dei coefficienti è **tridiagonale**. Questo tipo di sistemi compare frequentemente nella discretizzazione di equazioni differenziali mediante metodi alle differenze finite.

Consideriamo dunque il seguente sistema di equazioni lineari:

$$\begin{aligned}
v_1\phi_1 + w_1\phi_2 \qquad\qquad &= S_1 \\
u_2\phi_1 + v_2\phi_2 + w_2\phi_3 \qquad &= S_2 \\
\dots \quad u_3\phi_2 + v_3\phi_3 + w_3\phi_4 &= S_3 \\
\dots
\end{aligned}$$

dove u_i rappresentano i coefficienti sotto la diagonale principale, v_i rappresentano i coefficienti sulla diagonale principale, w_i rappresentano i coefficienti sopra la diagonale principale, S_i rappresentano i termini noti del sistema.

Eliminazione della prima variabile

Il primo passo dell'algoritmo consiste nell'eliminare progressivamente le variabili procedendo lungo il sistema.

Si sottrae la seconda equazione, moltiplicata per v_1, dalla prima moltiplicata per u_2.

Successivamente si procede a normalizzare le equazioni dividendo la prima equazione per v_1 e la seconda per $(v_2v_1 - u_2w_1)$.

In questo modo si ottiene un sistema equivalente nel quale i coefficienti delle variabili ϕ_1 e ϕ_2 risultano normalizzati a uno:

$$\begin{aligned}
\phi_1 + \frac{w_1}{v_1}\phi_2 \qquad\qquad &= \frac{S_1}{v_1} \\
\phi_2 + \frac{w_2v_1}{(v_2v_1 - u_2w_1)}\phi_3 &= \frac{S_2v_1 - S_1u_2}{(v_2v_1 - u_2w_1)} \\
u_3\phi_2 + v_3\phi_3 + w_3\phi_4 \qquad &= S_3
\end{aligned}$$

Riscrittura del sistema

Le espressioni precedenti possono essere riscritte in una forma più compatta osservando che $\frac{w_2 v_1}{v_2 v_1 - u_2 w_1} = \frac{w_2}{v_2 - u_2 \frac{w_1}{v_1}}$ e analogamente per il termine noto $\frac{S_2 - S_1 u_2/v_1}{(v_2 - u_2 w_1/v_1)}$.

Il sistema diventa quindi:

$$\begin{aligned}
\phi_1 + \frac{w_1}{v_1}\phi_2 &= \frac{S_1}{v_1} \\
\phi_2 + \frac{w_2}{\left(v_2 - u_2 \frac{w_1}{v_1}\right)}\phi_3 &= \frac{S_2 - S_1 \frac{u_2}{v_1}}{\left(v_2 - u_2 \frac{w_1}{v_1}\right)} \\
u_3\phi_2 + v_3\phi_3 + w_3\phi_4 &= S_3
\end{aligned}$$

Introduzione dei coefficienti ricorsivi

A questo punto è conveniente introdurre nuovi coefficienti che permettono di riscrivere il sistema in forma ricorsiva.

Definiamo:

$$\alpha_1 = \frac{w_1}{v_1} \quad , \qquad \beta_1 = \frac{S_1}{v_1}$$

e successivamente:

$$\alpha_2 = \frac{w_2}{(v_2 - u_2\alpha_1)} \quad , \qquad \beta_2 = \frac{S_2 - u_2\beta_1}{(v_2 - u_2\alpha_1)}$$

Utilizzando queste definizioni il sistema assume la forma

$$\begin{aligned}
\phi_1 + \alpha_1\phi_2 &= \beta_1 \\
\phi_2 + \alpha_2\phi_3 &= \beta_2 \\
u_3\phi_2 + v_3\phi_3 + w_3\phi_4 &= S_3 \\
&\dots
\end{aligned}$$

Passo ricorsivo

Il procedimento può essere ripetuto anche per le equazioni successive. Si sottrae dall'ultima equazione scritta la precedente moltiplicata per u_3, poi si divide il risultato per il coefficiente di ϕ_3. Si ottiene:

$$\phi_3 + \frac{w_3}{v_3 - u_3\alpha_2}\phi_4 = \frac{S_3 - u_3\beta_2}{v_3 - u_3\alpha_2}$$

Confrontando questa espressione con le definizioni precedenti si osserva che è possibile introdurre una definizione ricorsiva generale dei coefficienti.

Per $i = 2, \dots, N$:

$$\alpha_i = \frac{w_i}{(v_i - u_i\alpha_{i-1})} \tag{16}$$

$$\beta_i = \frac{S_i - u_i\beta_{i-1}}{(v_i - u_i\alpha_{i-1})} \tag{17}$$

Queste relazioni permettono di determinare progressivamente tutti i coefficienti α_i e β_i.

Forma finale del sistema

Dopo aver applicato questo procedimento a tutte le equazioni del sistema, assume la forma

$$\begin{aligned}
\phi_1 &= \beta_1 - \alpha_1\phi_2 \\
\phi_2 &= \beta_2 - \alpha_2\phi_3 \\
&\dots \\
\phi_{N-1} &= \beta_{N-1} - \alpha_{N-1}\phi_N \\
\phi_N &= \beta_N
\end{aligned}$$

L'ultima equazione fornisce immediatamente il valore di ϕ_N.

Una volta noto ϕ_N è possibile determinare tutte le altre incognite tramite una procedura di sostituzione all'indietro: $\phi_{N-1}, \phi_{N-2}, \dots, \phi_1$.

Costo computazionale

Il numero di operazioni in virgola mobile (floating point operations, FLOP) richiesto dall'algoritmo di Thomas per risolvere un sistema lineare di N equazioni è $\boxed{10N - 11}$.

Questo significa che il costo computazionale cresce linearmente con la dimensione del sistema.

Prestazioni

Consideriamo ad esempio un processore in grado di eseguire 600 GFLOPS cioè 600×10^9.

operazioni floating point al secondo (come ad esempio il processore AMD Ryzen Threadripper 2990WX).

In queste condizioni il tempo necessario per risolvere un sistema lineare con $N = 1000$ equazioni è dell'ordine di 17 ns .

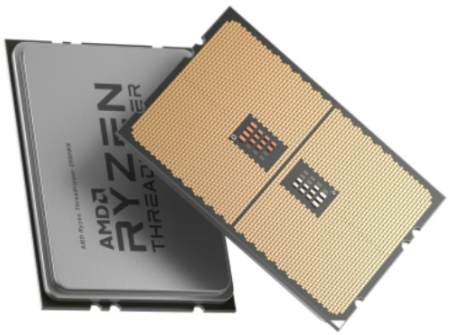


Fig. B.1: catzo e palle

Appendice C

Metodo di Jacobi

Introduzione

Consideriamo il sistema lineare

$$A\boldsymbol{\phi} = \boldsymbol{s} \tag{C.1}$$

dove $A \in \mathbb{R}^{n\times n}$ è una matrice assegnata, $\boldsymbol{\phi}$ il vettore incognito e $\boldsymbol{s}$ il termine noto.

Il metodo di Jacobi si basa sulla decomposizione della matrice A nella forma

$$A = D - E \tag{C.2}$$

dove: D è la matrice diagonale contenente gli elementi diagonali di A mentre E contiene l'opposto degli elementi fuori diagonale di A.

Sostituendo nella relazione iniziale si ottiene:

$$D\boldsymbol{\phi} = E\boldsymbol{\phi} + \boldsymbol{s} \tag{C.3}$$

Assumendo che D sia invertibile (cioè $a_{ii} \neq 0$ per ogni i), si può scrivere

$$\boldsymbol{\phi} = D^{-1}E\boldsymbol{\phi} + D^{-1}\boldsymbol{s} \tag{C.4}$$

Introducendo le definizioni

$$H = D^{-1}E, \qquad \boldsymbol{q} = D^{-1}\boldsymbol{s} \tag{C.5}$$

il sistema assume la forma

$$\boldsymbol{\phi} = H\boldsymbol{\phi} + \boldsymbol{q} \tag{C.6}$$

Struttura della matrice di iterazione

Dalla struttura di D ed E si ottiene che la matrice H ha forma

$$H = \begin{pmatrix} 0 & -\frac{a_{12}}{a_{11}} & -\frac{a_{13}}{a_{11}} & \cdots & -\frac{a_{1n}}{a_{11}} \\ -\frac{a_{21}}{a_{22}} & 0 & -\frac{a_{23}}{a_{22}} & \cdots & -\frac{a_{2n}}{a_{22}} \\ \vdots & \vdots & \vdots & & \vdots \\ -\frac{a_{n1}}{a_{nn}} & -\frac{a_{n2}}{a_{nn}} & -\frac{a_{n3}}{a_{nn}} & \cdots & 0 \end{pmatrix} \tag{C.7}$$

Gli elementi diagonali sono nulli.

Se la matrice A è diagonalmente dominante, vale

$$\sum_{j=1}^{n} |h_{ij}| < 1 \quad \forall i \tag{C.8}$$

da cui segue

$$\|H\|_\infty < 1 \quad \Rightarrow \quad \rho(H) < 1 \tag{C.9}$$

Quindi la matrice H è convergente.

Procedimento iterativo

Si introduce il processo iterativo

$$\boldsymbol{\phi}^{(m+1)} = H\boldsymbol{\phi}^{(m)} + \boldsymbol{q} \tag{C.10}$$

con condizione iniziale

$$\boldsymbol{\phi}^{(1)} = \boldsymbol{q} \tag{C.11}$$

Esplicitando le iterazioni:

$$\boldsymbol{\phi}^{(2)} = H\boldsymbol{\phi}^{(1)} + \boldsymbol{q} = H\boldsymbol{q} + \boldsymbol{q} \tag{C.12}$$

$$\boldsymbol{\phi}^{(3)} = H^2\boldsymbol{q} + H\boldsymbol{q} + \boldsymbol{q} \tag{C.13}$$

$$\vdots \tag{C.14}$$

$$\boldsymbol{\phi}^{(m+1)} = H^m\boldsymbol{q} + H^{m-1}\boldsymbol{q} + \cdots + \boldsymbol{q} \tag{C.15}$$

Convergenza del metodo

Moltiplicando per $(I - H)$ si ottiene

$$(I - H)\boldsymbol{\phi}^{(m+1)} = (I - H^{m+1})\boldsymbol{q} \tag{C.16}$$

e quindi

$$\boldsymbol{\phi}^{(m+1)} = (I - H)^{-1}(I - H^{m+1})\boldsymbol{q} \tag{C.17}$$

Passando al limite

$$\lim_{m\to\infty} \boldsymbol{\phi}^{(m+1)} = (I - H)^{-1}\boldsymbol{q} \tag{C.18}$$

D'altra parte, la soluzione esatta soddisfa

$$\boldsymbol{\phi} = H\boldsymbol{\phi} + \boldsymbol{q} \tag{C.19}$$

da cui

$$(I - H)\boldsymbol{\phi} = \boldsymbol{q} \quad \Rightarrow \quad \boldsymbol{\phi} = (I - H)^{-1}\boldsymbol{q} \tag{C.20}$$

Quindi

$$\lim_{m\to\infty} \boldsymbol{\phi}^{(m)} = \boldsymbol{\phi} \tag{C.21}$$

Studio della convergenza tramite autovalori

Sia H simmetrica. Allora esiste una matrice ortogonale U tale che

$$U^T H U = \Lambda \tag{C.22}$$

dove $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$.

Segue che

$$H\boldsymbol{u}_j = \lambda_j \boldsymbol{u}_j \tag{C.23}$$

Gli autovettori formano una base ortonormale.

C.0.1 Errore

Definiamo l'errore

$$\boldsymbol{\varepsilon}^{(m)} = \boldsymbol{\phi}^{(m)} - \boldsymbol{\phi} \tag{C.24}$$

Si ha

$$\boldsymbol{\varepsilon}^{(m)} = H^m \boldsymbol{\varepsilon}^{(0)} \tag{C.25}$$

Sviluppando sull'autobase:

$$\boldsymbol{\varepsilon}^{(0)} = \sum_{h=1}^{n} c_h^{(0)} \boldsymbol{\phi}_h \tag{C.26}$$

$$\boldsymbol{\varepsilon}^{(m)} = \sum_{h=1}^{n} c_h^{(0)} \lambda_h^m \boldsymbol{\phi}_h \tag{C.27}$$

La convergenza avviene se e solo se

$$|\lambda_h| < 1 \quad \forall h \tag{C.28}$$

cioè

$$\rho(H) < 1 \tag{C.29}$$

Condizioni di convergenza

Una condizione sufficiente per la convergenza è che la matrice A sia strettamente diagonalmente dominante:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \tag{C.30}$$

Oppure irriducibilmente diagonalmente dominante:

$$|a_{ii}| \geq \sum_{j \neq i} |a_{ij}| \tag{C.31}$$

con almeno una riga per cui la disuguaglianza è stretta.

Definizione di irriducibilità

Una matrice è riducibile se esiste una matrice di permutazione P tale che

$$PAP^T = \begin{pmatrix} \hat{A}_{11} & \hat{A}_{12} & \cdots \\ 0 & \hat{A}_{22} & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix} \tag{C.32}$$

Se ciò non è possibile, la matrice è detta irriducibile.

Una matrice è irriducibile se e solo se il suo grafo orientato è fortemente connesso.

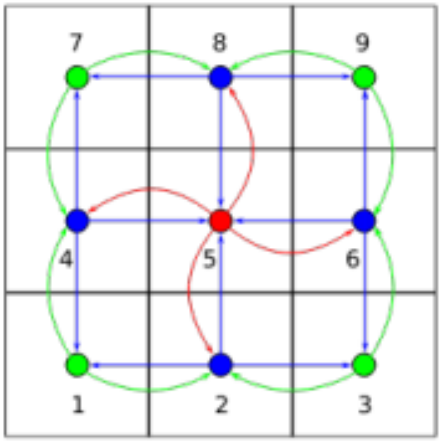


Fig. C.1: catzo e palle

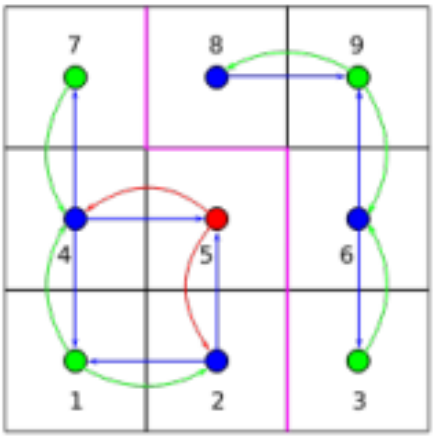


Fig. C.2: catzo e palle

Schema esplicito del metodo di Jacobi

Il sistema

$$\sum_{j=1}^{n} a_{ij}\phi_j = b_i \tag{C.33}$$

viene risolto iterativamente come

$$\phi_i^{(m+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} \phi_j^{(m)} \right) \tag{C.34}$$

Implementazione per equazione di diffusione

Nel caso bidimensionale:

$$-\frac{D}{h_x^2}(\phi_{i+1,j} - 2\phi_{i,j} + \phi_{i-1,j}) - \frac{D}{h_y^2}(\phi_{i,j+1} - 2\phi_{i,j} + \phi_{i,j-1}) + \Sigma_a \phi_{i,j} = Q_{i,j} \tag{C.35}$$

Definendo

$$W_{i,j} = E_{i,j} = \frac{D/h_x^2}{\Sigma_a + 2D/h_x^2 + 2D/h_y^2} \tag{C.36}$$

$$S_{i,j} = N_{i,j} = \frac{D/h_y^2}{\Sigma_a + 2D/h_x^2 + 2D/h_y^2} \tag{C.37}$$

$$q_{i,j} = \frac{Q_{i,j}}{\Sigma_a + 2D/h_x^2 + 2D/h_y^2} \tag{C.38}$$

lo schema diventa

$$\phi_{i,j}^{(m+1)} = W_{i,j}\phi_{i-1,j}^{(m)} + E_{i,j}\phi_{i+1,j}^{(m)} + S_{i,j}\phi_{i,j-1}^{(m)} + N_{i,j}\phi_{i,j+1}^{(m)} + q_{i,j} \tag{C.39}$$

Velocità di convergenza

L'errore si comporta come

$$\|\varepsilon^{(m)}\| \approx C\rho(H)^m \tag{C.40}$$

e vale

$$\lim_{m\to\infty} \frac{\|\varepsilon^{(m+1)}\|}{\|\varepsilon^{(m)}\|} = \rho(H) \tag{C.41}$$

Il tasso asintotico di convergenza è

$$R_\infty(H) = -\ln \rho(H) \tag{C.42}$$

Numero di iterazioni

Per ridurre l'errore di un fattore σ:

$$\rho(H)^n < \sigma \tag{C.43}$$

da cui

$$n > \frac{\ln \sigma}{\ln \rho(H)} = -\frac{\ln \sigma}{R_\infty(H)} \tag{C.44}$$

Osservazioni finali

Per avere una convergenza veloce è necessario che

$$\rho(H) \ll 1 \tag{C.45}$$

Si osserva che:

- se $\Sigma_a \to 0$, la dominanza diagonale diminuisce e $\rho(H) \to 1$;
- raffinando la griglia $(h_x, h_y \to 0)$, $\rho(H) \to 1$;

quindi una maggiore risoluzione spaziale comporta una convergenza più lenta.

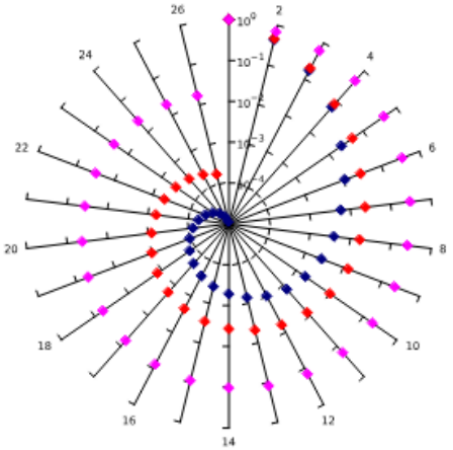


Fig. C.3: catzo e palle

Appendice D

Gauss-Siedel, SOR & LOR

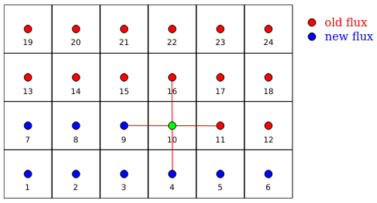


Fig. D.1

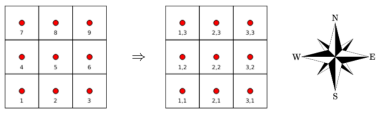


Fig. D.2

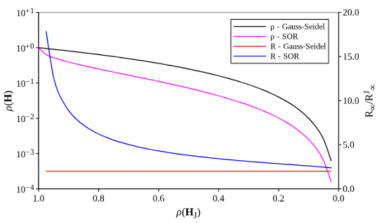


Fig. D.3

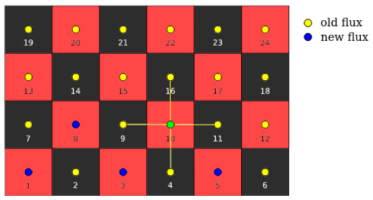


Fig. D.4

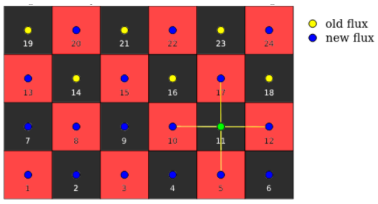


Fig. D.5

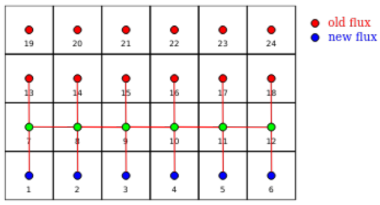


Fig. D.6

Appendice E

Alternate direction implicit

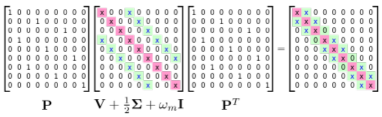


Fig. E.1

Appendice F

Metodi del gradiente

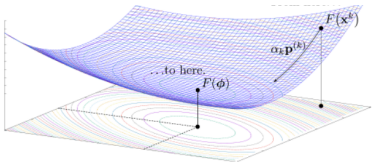


Fig. F.1

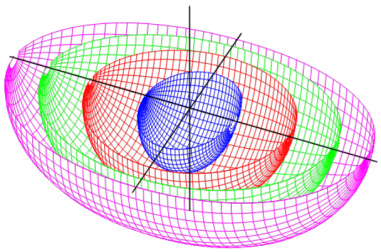


Fig. F.2

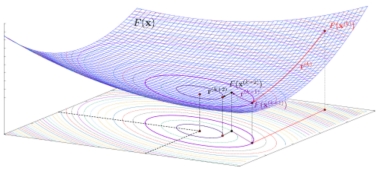


Fig. F.3

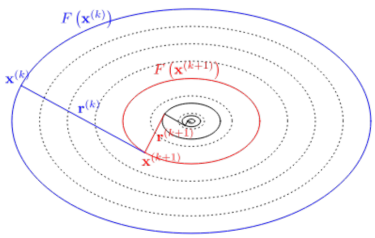


Fig. F.4

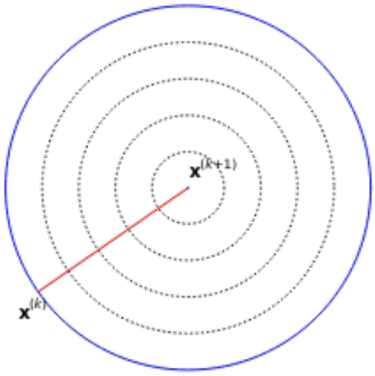


Fig. F.5

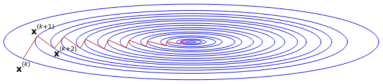


Fig. F.6